

ICPC TRD

Main (1), Graphs (1), Trees (10), Range Queries (18), Strings (24), Dynamic Programming (28), Mathematics (30), Geometry (35), Bit Operations (39), Extra (40),

Main

```
// #pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

#define fastio ios::sync_with_stdio(false); cin.tie(0); cout.tie(0)
#define all(v) v.begin(), v.end()
#define rall(v) v.rbegin(), v.rend()
#define pb push_back
#define fi first
#define se second

typedef long long ll;
typedef long double ld;
typedef pair<int,int> pii;
typedef vector<int> vi;
typedef vector<ll> vll;

const int INF = 1e9;
const ll LINF = 1e18;
const int MOD = 1e9+7;
const double EPS = 1e-9;

void solve(){}

int main() {
    #ifdef ICESI
        freopen("input.txt", "r", stdin);
    #endif
    fastio;
    int t = 1;
    // cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

Graphs

```
/**
 * Uso[1]: dx4, dy4, dx8, dy8, isValid(x, y, R, C);
 * Direcciones de movimiento en grillas 4/8-conexas
 * y validacion de coordenadas dentro de la
 * grilla.
 * Complejidad: O(1).
 */
const int dx4[4] = {1, -1, 0, 0};
const int dy4[4] = {0, 0, 1, -1};
const int dx8[8] = {1, 1, 0, -1, -1, -1, 0, 1};
const int dy8[8] = {0, 1, 1, 1, 0, -1, -1, -1};
#define isValid(x, y, R, C) ((x) >= 0 && (x) < (R)
    && (y) >= 0 && (y) < (C))

/**
 * Uso: Edge<T> e = {from, to, id, weight};
 * Arista con extremos, identificador opcional y
 * peso generico; usada internamente por Graph.
```

```
* Complejidad: O(1) por arista.
*/
template <typename T = ll> struct Edge { int from,
    to, id; T weight; };

/**
 * Uso[0]: DSU dsu(n); dsu.find(x); dsu.unite(a, b);
 * dsu.component_size(x); dsu.comps; dsu.
 * get_components();
 * Union-find con union por tamaño y acceso a los
 * miembros de cada componente.
 * Complejidad: find y unite casi O(α(N)) amortizado
 */
struct DSU {
    vector<int> p, sz;
    vector<vector<int>> members;
    int comps;
    DSU(int n) : p(n), sz(n, 1), members(n), comps(n) {
        iota(all(p), 0);
        for (int i = 0; i < n; i++) members[i].pb(i);
    }
    int find(int x) { return p[x] == x ? x : p[x] =
        find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        members[a].insert(members[a].end(), all(members[b]));
        vector<int>().swap(members[b]);
        comps--;
        return true;
    }
    const vector<int>& component_nodes(int x) {
        return members[find(x)];
    }
    int component_size(int x) { return sz[find(x)]; }
    vector<vector<int>> get_components(int first = 0)
    {
        const
        vector<vector<int>> res;
        for (int i = first; i < p.size(); i++) if (p[i]
            == i && !members[i].empty()) res.pb(members[i]);
        return res;
    }
};

/**
 * Uso[1]: Graph<ll> g(n); g.add_undirected_edge(u,
 * v, w); g.add_directed_edge(u, v, w, id);
 * Grafo con lista de adyacencia de índices a
 * aristas y pesos genericos.
 * Complejidad: agregar arista O(1).
 */
template <typename T = ll> struct Graph {
    int n;
    vector<vector<int>> adj; // indices a edges
    vector<Edge<T>> edges;
    Graph(int n) : n(n) { adj.resize(n + 1); }
    void add_directed_edge(int u, int v, T w = 1, int
```

```

        id = -1) {
        adj[u].pb(edges.size());
        edges.pb({u, v, id, w});
    }
    void add_undirected_edge(int u, int v, T w = 1,
        int id = -1) {
        add_directed_edge(u, v, w, id);
        add_directed_edge(v, u, w, id);
    }
};

/**
 * Uso[1]: Dijkstra<ll> di;
 * auto [ dist, par ] = di.run(g, src,
 * invalid_edges);
 * auto path = di.restore_path(target);
 * Caminos minimos desde un origen con pesos no
 * negativos.
 * Complejidad:  $O(E \log V)$ .
 */
template <typename T = ll> struct Dijkstra {
    vector<T> dist;
    vi p;
    pair<vector<T>, vi> run(const Graph<T>& g, int
        src, const vector<bool>& invalid_edges = {}) {
        dist.assign(g.n + 1, LINF);
        p.assign(g.n + 1, -1);
        priority_queue<pair<T, int>, vector<pair<T, int>
            >>, greater<>> pq;
        dist[src] = 0; pq.push({0, src});

        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;

            for (int idx : g.adj[u]) {
                auto& e = g.edges[idx];

                if (!invalid_edges.empty() && e.id != -1 &&
                    invalid_edges[e.id]) continue;

                if (dist[u] + e.weight < dist[e.to]) {
                    dist[e.to] = dist[u] + e.weight;
                    p[e.to] = u;
                    pq.push({dist[e.to], e.to});
                }
            }
        }
        return {dist, p};
    }
    vi restore_path(int target) {
        vi path;
        if (dist.empty() || dist[target] == LINF)
            return path;
        for (int v = target; v != -1; v = p[v]) path.pb
            (v);
        reverse(all(path));
        return path;
    }
};

/**
 * Uso[1]: auto dist = ZeroOneBfs<ll>().run(g, src)
 * ;
 * Caminos minimos con pesos restringidos a {0,1}
 * mediante un deque.
 * Complejidad:  $O(V + E)$ .

```

```

 */
template <typename T = ll> struct ZeroOneBfs {
    vector<T> run(const Graph<T>& g, int src) {
        vector<T> dist(g.n + 1, LINF);
        deque<int> dq;
        dist[src] = 0; dq.pb(src);
        while (!dq.empty()) {
            int u = dq.front(); dq.pop_front();
            for (int id : g.adj[u]) {
                auto& e = g.edges[id];
                if (dist[u] + e.weight < dist[e.to]) {
                    dist[e.to] = dist[u] + e.weight;
                    e.weight ? dq.pb(e.to) : dq.push_front(e.
                        to);
                }
            }
        }
        return dist;
    }
};

/**
 * Uso[1]: auto [es_bipartito, colores] = Bipartite
 * <ll>().check(g);
 * Verifica si el grafo es bipartito y colorea cada
 * componente con 0/1.
 * Complejidad:  $O(V + E)$ .
 */
template <typename T = ll> struct Bipartite {
    pair<bool, vi> check(const Graph<T>& g) {
        vi color(g.n + 1, -1);
        bool ok = true;
        for (int i = 1; i <= g.n && ok; i++) if (color[i]
            == -1) {
            queue<int> q;
            q.push(i); color[i] = 0;
            while (!q.empty()) {
                int u = q.front(); q.pop();
                for (int id : g.adj[u]) {
                    int v = g.edges[id].to;
                    if (color[v] == -1) color[v] = color[u] ^
                        1; q.push(v);
                    else if (color[v] == color[u]) ok = false
                        ;
                }
            }
        }
        return {ok, color};
    }
};

/**
 * Uso[1]: auto dist = FloydMarshall<ll>().run(g);
 * Caminos minimos entre todos los pares de nodos.
 * Complejidad:  $O(V^3)$  tiempo,  $O(V^2)$  espacio.
 */
template <typename T = ll> struct FloydMarshall {
    vector<vector<T>> run(const Graph<T>& g) {
        vector<vector<T>> dist(g.n + 1, vector<T>(g.n +
            1, LINF));
        for (int i = 1; i <= g.n; i++) dist[i][i] = 0;
        for (auto& e : g.edges) dist[e.from][e.to] =
            min(dist[e.from][e.to], e.weight);
        for (int k = 1; k <= g.n; k++)
            for (int i = 1; i <= g.n; i++)
                for (int j = 1; j <= g.n; j++)

```

```

        if (dist[i][k] < LINF && dist[k][j] <
            LINF) dist[i][j] = min(dist[i][j], dist[i][k] +
            dist[k][j]);
        return dist;
    }
};

/**
 * Uso[1]: auto order = TopoSort<ll>().run(g);
 * Orden topologico de un grafo dirigido aciclico;
 * devuelve vacio si existe un ciclo.
 * Complejidad:  $O(V + E)$ .
 */
template <typename T = ll> struct TopoSort {
    vi run(const Graph<T>& g) {
        vi in(g.n + 1, 0), order;
        for (auto& e : g.edges) in[e.to]++;
        queue<int> q;
        for (int i = 1; i <= g.n; i++) if (!in[i]) q.
            push(i);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            order.pb(u);
            for (int id : g.adj[u]) if (--in[g.edges[id].
                to] == 0) q.push(g.edges[id].to);
        }
        return order.size() == g.n ? order : vi();
    }
};

/**
 * Uso[1]: auto comp = TarjanSCC<ll>().run(g);
 * Componentes fuertemente conexas mediante Tarjan;
 * devuelve el id de componente de cada nodo.
 * Complejidad:  $O(V + E)$ .
 */
template <typename T = ll> struct TarjanSCC {
    vi run(const Graph<T>& g) {
        int n = g.n, timer = 0, ncomps = 0;
        vi val(n + 1, 0), comp(n + 1, -1), stk;
        auto dfs = [&](auto&& self, int u) -> int {
            int low = val[u] = ++timer;
            stk.pb(u);
            for (int id : g.adj[u]) {
                int v = g.edges[id].to;
                if (comp[v] < 0) low = min(low, val[v] ?
                    val[v] : self(self, v));
            }
            if (low == val[u]) {
                int x;
                do { x = stk.back(); stk.pop_back(); comp[x]
                    = ncomps; } while (x != u);
                ncomps++;
            }
            return val[u] = low;
        };
        for (int i = 1; i <= n; i++) if (!val[i]) dfs(
            dfs, i);
        return comp;
    }
};

/**
 * Uso[1]: auto [sin_ciclo_neg, dist, parent, ciclo] = BellmanFord<ll>().run(g, src);
 * Caminos minimos con pesos negativos; detecta y
 * recupera un ciclo negativo si existe.

```

```

 * Complejidad:  $O(V \cdot E)$ .
 */
template <typename T = ll> struct BellmanFord {
    tuple<bool, vector<T>, vi, vi> run(const Graph<T>
        & g, int src) {
        vector<T> dist(g.n + 1, LINF);
        vi p(g.n + 1, -1);
        dist[src] = 0;
        int x = -1;
        for (int i = 0; i < g.n; i++) {
            x = -1;
            for (auto& e : g.edges) if (dist[e.from] <
                LINF && dist[e.to] > dist[e.from] + e.weight) {
                dist[e.to] = max(-LINF, dist[e.from] + e.
                    weight);
                p[e.to] = e.from; x = e.to;
            }
        }
        vi cycle;
        bool ok = true;
        if (x != -1) {
            ok = false;
            int y = x;
            for (int i = 0; i < g.n; i++) y = p[y];
            for (int cur = y;; cur = p[cur]) { cycle.pb(
                cur); if (cur == y && cycle.size() > 1) break;
            }
            reverse(all(cycle));
        }
        return {ok, dist, p, cycle};
    }
};

/**
 * Uso[1]: auto ciclo = DirectedCycle<ll>().find(g);
 * Encuentra un ciclo en un grafo dirigido (vacio
 * si no existe).
 * Complejidad:  $O(V + E)$ .
 */
template <typename T = ll> struct DirectedCycle {
    vi find(const Graph<T>& g) {
        int n = g.n;
        vi state(n + 1, 0), parent(n + 1, -1), cycle;
        auto dfs = [&](auto&& self, int u) -> bool {
            state[u] = 1;
            for (int id : g.adj[u]) {
                int v = g.edges[id].to;
                if (state[v] == 0) {
                    parent[v] = u;
                    if (self(self, v)) return true;
                } else if (state[v] == 1) {
                    cycle.pb(v);
                    for (int cur = u; cur != -1 && cur != v;
                        cur = parent[cur]) cycle.pb(cur);
                    cycle.pb(v);
                    reverse(all(cycle));
                    return true;
                }
            }
            state[u] = 2;
            return false;
        };
        for (int i = 1; i <= n; i++) if (!state[i] &&
            dfs(dfs, i)) return cycle;
        return cycle;
    }
};

```

```

};

/**
 * Uso[1]: auto ciclo = UndirectedCycle<ll>().find(g);
 * Encuentra un ciclo en un grafo no dirigido (vacío si no existe).
 * Complejidad:  $O(V + E)$ .
 */
template <typename T = ll> struct UndirectedCycle {
    vi find(const Graph<T>& g) {
        int n = g.n;
        vi state(n + 1, 0), parent(n + 1, -1), cycle;
        auto dfs = [&](auto&& self, int u, int pe) -> bool {
            state[u] = 1;
            for (int id : g.adj[u]) {
                if ((id ^ 1) == pe) continue;
                int v = g.edges[id].to;
                if (state[v] == 0) {
                    parent[v] = u;
                    if (self(self, v, id)) return true;
                } else if (state[v] == 1) {
                    cycle.pb(v);
                    for (int cur = u; cur != -1 && cur != v; cur = parent[cur]) cycle.pb(cur);
                    cycle.pb(v);
                    reverse(all(cycle));
                    return true;
                }
            }
            state[u] = 2;
            return false;
        };
        for (int i = 1; i <= n; i++) if (!state[i] && dfs(dfs, i, -1)) return cycle;
        return cycle;
    }
};

/**
 * Uso[1]: auto [peso, aristas_mst] = Kruskal<ll>().run(g);
 * Arbol de expansion minima (MST) mediante Kruskal; devuelve el peso total y los índices de las aristas.
 * Complejidad:  $O(E \log E)$ .
 */
template <typename T = ll> struct Kruskal {
    pair<T, vi> run(const Graph<T>& g) {
        vi idx(g.edges.size());
        iota(all(idx), 0);
        sort(all(idx), [&](int a, int b) { return g.edges[a].weight < g.edges[b].weight; });
        DSU dsu(g.n + 1);
        T total = 0;
        vi mst;
        for (int i : idx) {
            auto& e = g.edges[i];
            if (dsu.unite(e.from, e.to)) {
                total += e.weight;
                mst.pb(i);
                if (mst.size() == g.n - 1) break;
            }
        }
        return {total, mst};
    }
};

}

/**
 * Uso[1]: auto circuito = EulerianCircuit<ll>().find(g, start, undirected);
 * Camino o circuito euleriano: recorre cada arista exactamente una vez (vacío si no existe).
 * Un circuito euleriano existe si el grafo es conexo y, en no dirigido, todos los grados son pares, o en dirigido,  $in(u) = out(u)$  en cada nodo.
 * Complejidad:  $O(V + E)$ .
 */
template <typename T = ll> struct EulerianCircuit {
    vi find(const Graph<T>& g, int start = 1, bool undirected = true) {
        int n = g.n;
        vi in(n + 1, 0), out(n + 1, 0);
        int max_id = -1;
        for (auto& e : g.edges) {
            out[e.from]++; in[e.to]++;
            max_id = max(max_id, e.id);
        }
        for (int i = 1; i <= n; i++) {
            if (undirected ? (out[i] & 1) : (in[i] != out[i])) return {};
        }
        vector<bool> used(max_id + 1, false);
        vi circuit, head(n + 1, 0);
        auto dfs = [&](auto&& self, int u) -> void {
            while (head[u] < g.adj[u].size()) {
                auto& e = g.edges[g.adj[u][head[u]++]];
                if (e.id == -1 || !used[e.id]) {
                    if (e.id != -1) used[e.id] = true;
                    self(self, e.to);
                }
            }
            circuit.pb(u);
        };
        dfs(dfs, start);
        reverse(all(circuit));
        int need = undirected ? g.edges.size() / 2 + 1 : g.edges.size() + 1;
        if (circuit.size() != need && !g.edges.empty()) return {};
        return circuit;
    }
};

/**
 * Uso[1]: Hamiltonian h(n); h.add_edge(u, v); auto path = h.find_path(); auto cycle = h.find_cycle();
 * Camino hamiltoniano: visita cada vertice exactamente una vez; si existe, find_cycle devuelve un ciclo (el último nodo conecta con el primero). Vacío si no hay.
 * Un camino hamiltoniano es NP-completo en general, por eso se resuelve por backtracking.
 * Indexación: nodos 1-indexados.
 * Complejidad:  $O(N!)$  peor caso, práctico para  $N \leq 15$ .
 */
struct Hamiltonian {
    int n;
    vector<vi> adj;
    vector<bool> vis;
};

```

```

Hamiltonian(int n) : n(n), adj(n + 1), vis(n + 1,
    false) {}
void add_edge(int u, int v) { adj[u].pb(v); adj[v]
    .pb(u); }
bool bt(int u, int cnt, int start, bool cycle, vi
    & path) {
    if (cnt == n) {
        if (!cycle) return true;
        for (int v : adj[u]) if (v == start) return
            true;
        return false;
    }
    for (int v : adj[u]) if (!vis[v]) {
        vis[v] = true; path.pb(v);
        if (bt(v, cnt + 1, start, cycle, path))
            return true;
        path.pop_back(); vis[v] = false;
    }
    return false;
}
vi find(bool cycle) {
    for (int s = 1; s <= n; s++) {
        fill(all(vis), false);
        vi path = {s};
        vis[s] = true;
        if (bt(s, 1, s, cycle, path)) return path;
    }
    return {};
}
vi find_path() { return find(false); } // camino
    que usa todos los nodos
vi find_cycle() { return find(true); } // ciclo
    que usa todos los nodos
};

/**
 * Uso[1]: FunctionalGraph fg(n); fg.build_lifting(
    succ); fg.kth_successor(u, k); fg.decompose(
    succ);
 * Grafo funcional (cada nodo un unico sucesor): k-
    esimo sucesor con binary lifting y
    descomposicion en ciclos.
 * Complejidad: preproceso  $O(N \log K)$ , k-esimo
    sucesor  $O(\log K)$ .
 */
struct FunctionalGraph {
    int n, log_k;
    vector<vector<int>> up;
    vi succ, in_cycle, cycle_id, cycle_pos,
        cycle_size, dist_to_cycle;
    vector<vi> cycles;
    FunctionalGraph(int n, int log_k = 60) : n(n),
        log_k(log_k),
        up(n + 1, vector<int>(log_k, 0)), succ(n + 1,
            0), in_cycle(n + 1, 0),
        cycle_id(n + 1, -1), cycle_pos(n + 1, -1),
        cycle_size(n + 1, 0), dist_to_cycle(n + 1, 0)
        {}
    // Descompone el grafo en ciclos y arboles
    colgantes
    void decompose(const vi& _succ) {
        succ = _succ;
        vi state(n + 1, 0);
        cycles.clear();
        for (int i = 1; i <= n; i++) if (!state[i]) {
            int cur = i;
            vi path;

```

```

            while (cur >= 1 && cur <= n && state[cur] ==
                0) {
                state[cur] = 1;
                path.pb(cur);
                cur = succ[cur];
            }
            if (cur >= 1 && cur <= n && state[cur] == 1)
                {
                    vi cyc;
                    bool start = false;
                    for (int u : path) { if (u == cur) start =
                        true; if (start) cyc.pb(u); }
                    int cid = cycles.size(), csz = cyc.size();
                    for (int pos = 0; pos < csz; pos++) {
                        int u = cyc[pos];
                        in_cycle[u] = 1; cycle_id[u] = cid;
                        cycle_pos[u] = pos; cycle_size[u] = csz;
                        dist_to_cycle[u] = 0;
                    }
                    cycles.pb(cyc);
                }
            for (int u : path) state[u] = 2;
        }
        for (int i = 1; i <= n; i++) if (!in_cycle[i])
            {
                int cur = i;
                vi path;
                while (cur >= 1 && cur <= n && !in_cycle[cur]
                    && cycle_id[cur] == -1) path.pb(cur), cur =
                    succ[cur];
                int cid = (cur >= 1 && cur <= n) ? cycle_id[
                    cur] : -1;
                int csz = (cur >= 1 && cur <= n) ? cycle_size
                    [cur] : 0;
                int d = (cur >= 1 && cur <= n) ?
                    dist_to_cycle[cur] : 0;
                for (int j = path.size() - 1; j >= 0; j--) {
                    int u = path[j];
                    cycle_id[u] = cid; cycle_size[u] = csz;
                    dist_to_cycle[u] = ++d;
                }
            }
    }
    // Binary lifting sobre succ para responder k-
    esimo sucesor
    void build_lifting(const vi& _succ) {
        succ = _succ;
        for (int i = 1; i <= n; i++) up[i][0] = succ[i];
        for (int j = 1; j < log_k; j++)
            for (int i = 1; i <= n; i++) {
                int p = up[i][j - 1];
                up[i][j] = (p >= 1 && p <= n) ? up[p][j -
                    1] : 0;
            }
    }
    int kth_successor(int u, ll k) {
        for (int j = 0; j < log_k; j++) if (k & (1LL <<
            j)) {
            u = up[u][j];
            if (u < 1 || u > n) return 0;
        }
        return u;
    }
};

```

```

/**
 * Uso[1]: LCA<ll> lca(n); lca.build(root, g); lca.
 * lca(u, v); lca.dist(u, v);
 * Ancestro comun mas bajo y distancia en numero de
 * aristas mediante binary lifting, construido
 * directamente sobre la estructura Graph.
 * Indexación: nodos 1-indexados.
 * Complejidad: preproceso  $O(N \log N)$ , consulta
 *  $O(\log N)$ .
 */
template <typename T = ll> struct LCA {
    int n, log_n;
    vector<vi> up;
    vi depth;
    LCA(int n, int log_n = 20) : n(n), log_n(log_n),
        up(n + 1, vi(log_n, 0)), depth(n + 1, 0) {}
    void dfs(int u, int p, const Graph<T>& g) {
        up[u][0] = p;
        for (int j = 1; j < log_n; j++) up[u][j] = up[
            up[u][j - 1]][j - 1];
        for (int id : g.adj[u]) {
            int v = g.edges[id].to;
            if (v != p) depth[v] = depth[u] + 1, dfs(v, u
            , g);
        }
    }
    void build(int root, const Graph<T>& g) { dfs(
        root, root, g); }
    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        int k = depth[u] - depth[v];
        for (int j = 0; j < log_n; j++) if (k & (1 << j
        )) u = up[u][j];
        if (u == v) return u;
        for (int j = log_n - 1; j >= 0; j--) if (up[u][
            j] != up[v][j]) u = up[u][j], v = up[v][j];
        return up[u][0];
    }
    int dist(int u, int v) { return depth[u] + depth[
        v] - 2 * depth[lca(u, v)]; }
};

/**
 * Uso[1]: TwoSat s(n); s.add_clause(u, tu, v, tv);
 * s.force_value(u, t); s.solve();
 * Satisfacibilidad booleana 2-SAT con clausulas (
 * or, implicacion, xor, equivalencia) y la
 * asignacion resultante.
 * Complejidad:  $O(V + E)$ .
 */
struct TwoSat {
    int n;
    Graph<ll> G;
    vector<bool> assignment;
    TwoSat(int n) : n(n), G(2 * n), assignment(n + 1,
        false) {}
    int node(int u, bool t) { return t ? u : u + n; }
    void add_clause(int u, bool tu, int v, bool tv) {
        G.add_directed_edge(node(u, !tu), node(v, tv));
        G.add_directed_edge(node(v, !tv), node(u, tu));
    }
    void force_value(int u, bool t) { add_clause(u, t
        , u, t); }
    void add_implication(int u, bool tu, int v, bool
        tv) { add_clause(u, !tu, v, tv); }

```

```

    void add_xor(int u, bool tu, int v, bool tv) {
        add_clause(u, tu, v, tv);
        add_clause(u, !tu, v, !tv);
    }
    void add_equivalence(int u, bool tu, int v, bool
        tv) {
        add_implication(u, tu, v, tv);
        add_implication(v, tv, u, tu);
    }
    bool solve() {
        vi comp = TarjanSCC<ll>().run(G);
        for (int i = 1; i <= n; i++) {
            if (comp[i] == comp[i + n]) return false;
            assignment[i] = comp[i] < comp[i + n];
        }
        return true;
    }
};

/**
 * Uso[1]: BipartiteMatcher bm(n_izq, n_der); bm.
 * add_edge(u, v); bm.max_matching(); bm.match_of(
 * v);
 * Emparejamiento maximo bipartito con el algoritmo
 * de Kuhn; el resultado queda en match_of(v) =
 * vertice izquierdo que empareja a v (o -1).
 * Indexación: nodos 1-indexados.
 * Complejidad:  $O(V \cdot E)$ .
 */
struct BipartiteMatcher {
    int n, m;
    vector<vi> adj;
    vi match;
    BipartiteMatcher(int n, int m) : n(n), m(m), adj(
        n + 1), match(m + 1, -1) {}
    void add_edge(int u, int v) { adj[u].pb(v); }
    bool dfs(int u, vector<bool>& vis) {
        for (int v : adj[u]) if (!vis[v]) {
            vis[v] = true;
            if (match[v] < 0 || dfs(match[v], vis)) {
                match[v] = u; return true; }
        }
        return false;
    }
    int max_matching() {
        int ans = 0;
        for (int i = 1; i <= n; i++) {
            vector<bool> vis(m + 1, false);
            if (dfs(i, vis)) ans++;
        }
        return ans;
    }
    int match_of(int v) { return match[v]; } //
    vertice izquierdo emparejado a v, -1 si v esta
    libre
};

/**
 * Uso[1]: HopcroftKarp hk(n_izq, n_der); hk.
 * add_edge(u, v); hk.max_matching(); hk.match_of(
 * v);
 * Emparejamiento maximo bipartito con Hopcroft-
 * Karp; el resultado queda en match_of(v) =
 * vertice izquierdo que empareja a v (o 0).
 * Indexación: nodos 1-indexados.
 * Complejidad:  $O(E\sqrt{V})$ .

```

```

*/
struct HopcroftKarp {
    int n, m;
    vector<vi> adj;
    vi pairU, pairV, dist;
    HopcroftKarp(int n, int m) : n(n), m(m), adj(n +
        1), pairU(n + 1, 0), pairV(m + 1, 0), dist(n +
        1) {}
    void add_edge(int u, int v) { adj[u].pb(v); }
    bool bfs() {
        queue<int> q;
        for (int u = 1; u <= n; u++) {
            if (pairU[u]) dist[u] = INF;
            else dist[u] = 0, q.push(u);
        }
        dist[0] = INF;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (dist[u] < dist[0])
                for (int v : adj[u]) if (dist[pairV[v]] ==
                    INF) dist[pairV[v]] = dist[u] + 1, q.push(pairV
                    [v]);
        }
        return dist[0] != INF;
    }
    bool dfs(int u) {
        if (!u) return true;
        for (int v : adj[u]) if (dist[pairV[v]] == dist
            [u] + 1 && dfs(pairV[v])) {
            pairV[v] = u; pairU[u] = v; return true;
        }
        dist[u] = INF;
        return false;
    }
    int max_matching() {
        int res = 0;
        while (bfs()) for (int u = 1; u <= n; u++) if
            (!pairU[u] && dfs(u)) res++;
        return res;
    }
    int match_of(int v) { return pairV[v]; } //
    // vertice izquierdo emparejado a v, 0 si v esta
    // libre
};
/**
 * Uso[0]: MaxClique mc(n); mc.add_edge(u, v); mc.
 * solve();
 * Clique maxima de un grafo con hasta 60 nodos
 * usando bitsets.
 * Complejidad: exponencial en el peor caso,
 * practico para N ≤ 60.
 */
struct MaxClique {
    int n, max_sz;
    vector<ll> adj;
    MaxClique(int n) : n(n), max_sz(0), adj(n, 0) {}
    void add_edge(int u, int v) { adj[u] |= (1LL << v
        ); adj[v] |= (1LL << u); }
    void dfs(ll R, ll P, ll X) {
        if (!P && !X) { max_sz = max(max_sz,
            __builtin_popcountll(R)); return; }
        ll iter = P & ~adj[__builtin_ctzll(P | X)];
        while (iter) {
            int v = __builtin_ctzll(iter); ll v_bit = 1LL
                << v;
            dfs(R | v_bit, P & adj[v], X & adj[v]);
        }
    }
};

P &= ~v_bit; X |= v_bit; iter &= ~v_bit;
}
}
int solve() { dfs(0, (1LL << n) - 1, 0); return
    max_sz; }
};
/**
 * Uso[1]: BridgeTree bt(n, num_aristas); bt.
 * add_edge(u, v, id); bt.build();
 * Arbol de puentes: detecta puentes (aristas cuya
 * eliminacion desconecta el grafo) y puntos de
 * articulacion, y contrae cada componente 2-
 * arista-conexa en un nodo del arbol resultante (
 * tree_adj).
 * Indexación: nodos 1-indexados; aristas
 * identificadas por su id.
 * Complejidad: O(V + E).
 */
struct BridgeTree {
    int n, timer, num_comps;
    vector<vector<pair<int, int>>> adj;
    vi tin, low, comp_id;
    vector<bool> is_bridge, is_articulation;
    vector<vi> tree_adj;
    BridgeTree(int n, int num_edges) : n(n), adj(n +
        1), tin(n + 1, -1), low(n + 1, -1),
        comp_id(n + 1, 0), is_bridge(num_edges, false),
        is_articulation(n + 1, false), timer(0),
        num_comps(0) {}
    void add_edge(int u, int v, int id) { adj[u].pb({
        v, id}); adj[v].pb({u, id}); }
    void dfs_tarjan(int u, int p = -1) {
        tin[u] = low[u] = ++timer;
        int children = 0;
        for (auto [v, id] : adj[u]) {
            if (v == p) continue;
            if (tin[v] != -1) low[u] = min(low[u], tin[v
                ]);
            else {
                children++;
                dfs_tarjan(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) is_bridge[id] = true;
                if (low[v] >= tin[u] && p != -1)
                    is_articulation[u] = true;
            }
        }
        if (p == -1 && children > 1) is_articulation[u]
            = true;
    }
    void dfs_comp(int u, int cur) {
        comp_id[u] = cur;
        for (auto [v, id] : adj[u]) if (!comp_id[v]) {
            if (is_bridge[id]) {
                tree_adj.pb(vi());
                tree_adj[cur].pb(++num_comps);
                tree_adj[num_comps].pb(cur);
                dfs_comp(v, num_comps);
            } else dfs_comp(v, cur);
        }
    }
    void build() {
        for (int i = 1; i <= n; i++) if (tin[i] == -1)
            dfs_tarjan(i);
        tree_adj.pb(vi());
        for (int i = 1; i <= n; i++) if (!comp_id[i])

```

```

    tree_adj.pb(vi()), dfs_comp(i, ++num_comps);
}
};

/**
 * Uso[0]: Dinic d(n, s, t); d.add_edge(u, v, cap);
d.max_flow(); d.restore_path();
 * Flujo maximo con el algoritmo de Dinic, dirigido
o no dirigido; restore_path devuelve un camino
s → t por el que circula flujo positivo.
 * Indexación: nodos 0-indexados.
 * Complejidad: O(V2E) en general, O(E√V) en redes
unitarias.
 */
struct Dinic {
    struct FlowEdge { int u, v; ll cap, flow = 0; };
    int n, s, t;
    vector<FlowEdge> edges;
    vector<vi> adj;
    vi level, ptr;
    Dinic(int n, int s, int t) : n(n), s(s), t(t),
        adj(n + 1), level(n + 1), ptr(n + 1) {}
    int add_edge(int u, int v, ll cap, bool directed
        = true) {
        adj[u].pb(edges.size()); edges.pb({u, v, cap});
        adj[v].pb(edges.size()); edges.pb({v, u,
            directed ? 0 : cap});
        return (int)edges.size() - 2; // indice de la
            arista forward
    }
    bool bfs() {
        fill(all(level), -1);
        level[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : adj[u]) {
                auto& e = edges[id];
                if (e.cap - e.flow >= 1 && level[e.v] ==
                    -1) level[e.v] = level[u] + 1, q.push(e.v);
            }
        }
        return level[t] != -1;
    }
    ll dfs(int u, ll pushed) {
        if (!pushed) return 0;
        if (u == t) return pushed;
        for (int& cid = ptr[u]; cid < adj[u].size();
            cid++) {
            auto& e = edges[adj[u][cid]];
            ll tr = e.cap - e.flow;
            if (level[u] + 1 != level[e.v] || !tr)
                continue;
            ll push = dfs(e.v, min(pushed, tr));
            if (push) {
                e.flow += push;
                edges[adj[u][cid] ^ 1].flow -= push;
                return push;
            }
        }
        return 0;
    }
    ll max_flow() { return max_flow(s, t); }
    ll max_flow(int src, int snk) {
        s = src; t = snk;
        ll flow = 0;

```

```

        while (bfs()) {
            fill(all(ptr), 0);
            while (ll pushed = dfs(s, LINF)) flow +=
                pushed;
        }
        return flow;
    }
    vi restore_path() { // camino s->t por aristas
        con flujo positivo (vacío si no llega)
        vi path = {s};
        int u = s;
        while (u != t) {
            bool advanced = false;
            for (int id : adj[u]) if (edges[id].flow > 0)
            {
                u = edges[id].v; path.pb(u); advanced =
                    true; break;
            }
            if (!advanced) return {};
        }
        return path;
    }
};

/**
 * Uso[1]: DinicLB d(n); int id = d.add_edge(u, v,
lo, hi); ll flujo = d.max_flow(s, t); ll f = d.
edge_flow(id);
 * Flujo maximo con cotas inferiores (lower bounds)
en cada arista; devuelve -1 si no existe flujo
factible.
 * Complejidad: O(Dinic) sobre el grafo
transformado (dos fases de flujo maximo).
 */
struct DinicLB {
    int n, ss, tt;
    Dinic din;
    vector<ll> bal, lo;
    vector<int> edge_idx;
    bool built = false;
    ll total = 0;
    DinicLB(int n) : n(n), ss(0), tt(n + 1), din(n +
        2, 0, 0), bal(n + 2, 0) {}
    int add_edge(int u, int v, ll lower, ll upper) {
        bal[u] -= lower; bal[v] += lower;
        lo.pb(lower);
        int id = din.add_edge(u, v, upper - lower);
        edge_idx.pb(id);
        return (int)lo.size() - 1;
    }
    void build_super() {
        if (built) return;
        built = true;
        for (int i = 1; i <= n; i++) {
            if (bal[i] > 0) din.add_edge(ss, i, bal[i]),
                total += bal[i];
            else if (bal[i] < 0) din.add_edge(i, tt, -bal
                [i]);
        }
    }
    bool feasible() {
        build_super();
        return din.max_flow(ss, tt) == total;
    }
    ll max_flow(int s, int t) {
        int idx = din.add_edge(t, s, LINF);

```

```

    if (!feasible()) return -1;
    ll flow = din.edges[idx].flow; // flujo de la
    circulacion (valor base s->t)
    // elimina t->s (cap y flow en ambas direcciones
    , si no queda flujo residual cancelable)
    din.edges[idx].flow = 0; din.edges[idx].cap =
    0;
    din.edges[idx ^ 1].flow = 0; din.edges[idx ^
    1].cap = 0;
    return flow + din.max_flow(s, t);
}
ll edge_flow(int id) const { return lo[id] + din.
edges[edge_idx[id]].flow; }
};

/**
 * Uso[0]: MCMF m(n); m.add_edge(u, v, cap, costo);
 * auto [flujo, costo] = m.solve(s, t);
 * Flujo maximo de costo minimo (min cost max flow)
 * con potenciales para costos negativos.
 * Complejidad:  $O(F \cdot E \log V)$  con  $F$  el flujo total.
 */
struct MCMF {
    struct Edge { int to, rev; ll cap, flow, cost; };
    int n;
    vector<vector<Edge>> adj;
    vll dist, pot;
    vi p_node, p_edge;
    MCMF(int n) : n(n), adj(n + 1), dist(n + 1), pot(
    n + 1, 0), p_node(n + 1), p_edge(n + 1) {}
    void add_edge(int u, int v, ll cap, ll cost) {
        adj[u].pb({v, (int)adj[v].size(), cap, 0, cost
        });
        adj[v].pb({u, (int)adj[u].size() - 1, 0, 0, -
        cost});
    }
    // Inicializa potenciales si hay costos negativos
    . Omitir si todos son >= 0.
    void init_potentials(int s) {
        fill(all(pot), LINF);
        pot[s] = 0;
        for (int i = 0; i < n; i++)
            for (int u = 0; u <= n; u++) if (pot[u] !=
            LINF)
                for (auto& e : adj[u]) if (e.cap - e.flow >
                0 && pot[e.to] > pot[u] + e.cost) pot[e.to] =
                pot[u] + e.cost;
    }
    bool dijkstra(int s, int t) {
        fill(all(dist), LINF);
        dist[s] = 0;
        priority_queue<pair<ll, int>, vector<pair<ll,
        int>>, greater<>> pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (int i = 0; i < adj[u].size(); i++) {
                auto& e = adj[u][i];
                ll rc = e.cost + pot[u] - pot[e.to]; //
                costo reducido
                if (e.cap - e.flow > 0 && dist[e.to] > dist
                [u] + rc) {
                    dist[e.to] = dist[u] + rc;
                    p_node[e.to] = u; p_edge[e.to] = i;
                    pq.push({dist[e.to], e.to});
                }
            }
        }
        return dist[t] != LINF;
    }
}

pair<ll, ll> solve(int s, int t) { // {flujo,
costo}
    init_potentials(s);
    ll flow = 0, cost = 0;
    while (dijkstra(s, t)) {
        for (int i = 0; i <= n; i++) if (dist[i] !=
        LINF) pot[i] += dist[i];
        ll push = LINF;
        for (int cur = t; cur != s; cur = p_node[cur
        ])
            push = min(push, adj[p_node[cur]][p_edge[
            cur]].cap - adj[p_node[cur]][p_edge[cur]].flow
            );
        for (int cur = t; cur != s; cur = p_node[cur
        ]) {
            auto& e = adj[p_node[cur]][p_edge[cur]];
            e.flow += push;
            adj[cur][e.rev].flow -= push;
            cost += push * e.cost;
        }
        flow += push;
    }
    return {flow, cost};
}

/**
 * Uso[1]: DominatorTree dt(n); dt.add_edge(u, v);
 * dt.build(root);
 * Arbol dominador de un grafo dirigido desde una
 * raiz; dom[u] son los hijos de u en el arbol.
 * Complejidad:  $O((V + E) \log V)$ .
 */
struct DominatorTree {
    int n, t;
    vector<vi> adj, rev, dom;
    vi dfn, id, p, sdom, idom, dsu, best;
    DominatorTree(int n) : n(n), t(0), adj(n + 1),
    rev(n + 1), dom(n + 1),
    dfn(n + 1, 0), id(n + 1), p(n + 1), sdom(n + 1)
    , idom(n + 1), dsu(n + 1), best(n + 1) {}
    void add_edge(int u, int v) { adj[u].pb(v); }
    void dfs(int u) {
        dfn[u] = ++t;
        id[t] = u;
        sdom[t] = best[t] = dsu[t] = t;
        for (int v : adj[u]) {
            if (!dfn[v]) dfs(v), p[dfn[v]] = dfn[u];
            rev[dfn[v]].pb(dfn[u]);
        }
    }
    int find(int x) {
        if (x == dsu[x]) return x;
        int y = find(dsu[x]);
        if (sdom[best[x]] > sdom[best[dsu[x]]]) best[x]
        = best[dsu[x]];
        return dsu[x] = y;
    }
    void build(int root) {
        dfs(root);
        vector<vi> bucket(n + 1);

```

```

for (int i = t; i >= 2; i--) {
    for (int u : rev[i]) {
        find(u);
        if (sdom[best[u]] < sdom[i]) sdom[i] = sdom[best[u]];
    }
    bucket[sdom[i]].pb(i);
    dsu[i] = p[i];
    for (int u : bucket[p[i]]) {
        find(u);
        idom[u] = sdom[best[u]] == sdom[u] ? sdom[u] : best[u];
    }
    bucket[p[i]].clear();
}
for (int i = 2; i <= t; i++) {
    if (idom[i] != sdom[i]) idom[i] = idom[idom[i]];
    dom[id[idom[i]]].pb(id[i]);
}
};

```

Trees

```

/**
 * Uso[1]: TreeEdge<T> e = {to, id, weight};
 * Arista de arbol con destino, identificador
 * opcional y peso generico.
 * Complejidad: O(1) por arista.
 */
template <typename T = ll> struct TreeEdge { int to
, id; T weight; };

/**
 * Uso[1]: Tree<ll> t(n); t.add_edge(u, v, w); t.
 * init(root); t.get_diameter(); t.get_centers();
 * Arbol con raiz: tiempos de entrada/salida (euler
 * ), tamanos de subarbol, diametro con camino y
 * centros.
 * Las distancias mas lejanas de cada nodo se
 * calculan aparte con la funcion
 * all_farthest_distances(t).
 * Complejidad: init y diametro O(N).
 */
template <typename T = ll> struct Tree {
    int n, root, timer;
    vector<vector<TreeEdge<T>>> adj;
    vi parent, depth, sz, tin, tout, euler_order;
    vector<T> dist_from_root;
    Tree(int n = 0) : n(n), root(1), timer(0), adj(n
    + 1), parent(n + 1), depth(n + 1),
    sz(n + 1), tin(n + 1), tout(n + 1),
    dist_from_root(n + 1) {}
    void add_edge(int u, int v, T w = 1, int id = -1)
    {
        adj[u].pb({v, id, w});
        adj[v].pb({u, id, w});
    }
    void add_directed_edge(int u, int v, T w = 1, int
    id = -1) { adj[u].pb({v, id, w}); }
    void dfs_init(int u, int p = 0, int d = 0, T cur
    = 0) {
        parent[u] = p; depth[u] = d; dist_from_root[u]
        = cur;
        sz[u] = 1; tin[u] = ++timer;
        euler_order.pb(u);
    }
};

```

```

for (auto& e : adj[u]) if (e.to != p) {
    dfs_init(e.to, u, d + 1, cur + e.weight);
    sz[u] += sz[e.to];
}
tout[u] = timer;
}
void init(int root = 1) {
    this->root = root; timer = 0;
    euler_order.clear();
    dfs_init(root, 0, 0, 0);
}
struct DiameterResult { T length; int u, v; vi
path; };
DiameterResult get_diameter() const {
    auto farthest = [&](int start) {
        vector<T> dist(n + 1, -1);
        queue<int> q;
        q.push(start); dist[start] = 0;
        int far = start;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (dist[u] > dist[far]) far = u;
            for (auto& e : adj[u]) if (dist[e.to] ==
            -1) dist[e.to] = dist[u] + e.weight, q.push(e.
            to);
        }
        return pair<int, T>(far, dist[far]);
    };
    int u = farthest(1).fi;
    auto [v, length] = farthest(u);
    vi prev(n + 1, -1);
    queue<int> q;
    q.push(u); prev[u] = 0;
    while (!q.empty()) {
        int cur = q.front(); q.pop();
        if (cur == v) break;
        for (auto& e : adj[cur]) if (prev[e.to] ==
        -1) prev[e.to] = cur, q.push(e.to);
    }
    vi path;
    for (int cur = v; cur != 0; cur = prev[cur])
    path.pb(cur);
    reverse(all(path));
    return {length, u, v, path};
}
// 1 o 2 nodos centrales del diametro
vi get_centers() const {
    auto d = get_diameter();
    int m = d.path.size();
    if (m & 1) return {d.path[m / 2]};
    return {d.path[m / 2 - 1], d.path[m / 2]};
}
};

/**
 * Uso[0]: auto dist = all_farthest_distances(t);
 * Distancia maxima desde cada nodo a cualquier
 * otro nodo, usando los extremos del diametro
 * como preprocesamiento.
 * Indexación: 0-indexado (dist[i] para el nodo i).
 * Complejidad: O(N) (dos BFS sobre los extremos
 * del diametro).
 */
template <typename T = ll>
vector<T> all_farthest_distances(const Tree<T>& t)

```

```

{
    auto d = t.get_diameter();
    auto dist_from = [&](int start) {
        vector<T> dist(t.n + 1, -1);
        queue<int> q;
        q.push(start); dist[start] = 0;
        while (!q.empty()) {
            int cur = q.front(); q.pop();
            for (auto& e : t.adj[cur]) if (dist[e.to] == -1) dist[e.to] = dist[cur] + e.weight, q.push(e.to);
        }
        return dist;
    };
    auto du = dist_from(d.u), dv = dist_from(d.v);
    vector<T> res(t.n + 1);
    for (int i = 1; i <= t.n; i++) res[i] = max(du[i], dv[i]);
    return res;
}

/**
 * Uso[1]: BinaryLifting<ll> bl(t, root); bl.lca(u, v); bl.kth_ancestor(u, k); bl.path_sum(u, v); bl.dist_weighted(u, v);
 * Binary lifting sobre un arbol: lca, k-esimo ancestro, suma/max/min de aristas en el camino y distancias.
 * Complejidad: preproceso  $O(N \log N)$ , consulta  $O(\log N)$ .
 */
template <typename T = ll> struct BinaryLifting {
    int n, log_n;
    vi depth;
    vector<vi> up;
    vector<vector<T>> mx, mn, sum;
    vector<T> dist_root;
    BinaryLifting(const Tree<T>& tree, int root = 1) {
        n = tree.n;
        log_n = 32 - __builtin_clz(n) + 1;
        depth = tree.depth;
        dist_root = tree.dist_from_root;
        up.assign(n + 1, vi(log_n, 0));
        mx.assign(n + 1, vector<T>(log_n, -LINF));
        mn.assign(n + 1, vector<T>(log_n, LINF));
        sum.assign(n + 1, vector<T>(log_n, 0));
        for (int u = 1; u <= n; u++) up[u][0] = tree.parent[u] ? tree.parent[u] : u;
        for (int u = 1; u <= n; u++)
            for (auto& e : tree.adj[u]) if (e.to == tree.parent[u]) mx[u][0] = mn[u][0] = sum[u][0] = e.weight;
        for (int j = 1; j < log_n; j++)
            for (int i = 1; i <= n; i++) {
                int p = up[i][j - 1];
                up[i][j] = up[p][j - 1];
                mx[i][j] = max(mx[i][j - 1], mx[p][j - 1]);
                mn[i][j] = min(mn[i][j - 1], mn[p][j - 1]);
                sum[i][j] = sum[i][j - 1] + sum[p][j - 1];
            }
    }
    int kth_ancestor(int u, int k) {
        if (depth[u] < k) return 0;
        for (int j = 0; j < log_n; j++) if (k & (1 << j)) u = up[u][j];
        return u;
    }
};

```

```

}
int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    u = kth_ancestor(u, depth[u] - depth[v]);
    if (u == v) return u;
    for (int j = log_n - 1; j >= 0; j--) if (up[u][j] != up[v][j]) u = up[u][j], v = up[v][j];
    return up[u][0];
}
int dist_edges(int u, int v) { return depth[u] + depth[v] - 2 * depth[lca(u, v)]; }
T dist_weighted(int u, int v) { return dist_root[u] + dist_root[v] - 2 * dist_root[lca(u, v)]; }
// k-esimo nodo en el camino u->v (0-indexed: 0 = u)
int kth_node_on_path(int u, int v, int k) {
    int anc = lca(u, v);
    int d1 = depth[u] - depth[anc], d2 = depth[v] - depth[anc];
    if (k <= d1) return kth_ancestor(u, k);
    k -= d1;
    if (k <= d2) return kth_ancestor(v, d2 - k);
    return 0;
}
T path_max(int u, int v) {
    int anc = lca(u, v);
    T res = -LINF;
    auto lift = [&](int node, int steps) {
        for (int j = 0; j < log_n; j++) if (steps & (1 << j)) res = max(res, mx[node][j]), node = up[node][j];
    };
    lift(u, depth[u] - depth[anc]);
    lift(v, depth[v] - depth[anc]);
    return res;
}
T path_min(int u, int v) {
    int anc = lca(u, v);
    T res = LINF;
    auto lift = [&](int node, int steps) {
        for (int j = 0; j < log_n; j++) if (steps & (1 << j)) res = min(res, mn[node][j]), node = up[node][j];
    };
    lift(u, depth[u] - depth[anc]);
    lift(v, depth[v] - depth[anc]);
    return res;
}
T path_sum(int u, int v) {
    int anc = lca(u, v);
    T res = 0;
    auto lift = [&](int node, int steps) {
        for (int j = 0; j < log_n; j++) if (steps & (1 << j)) res += sum[node][j], node = up[node][j];
    };
    lift(u, depth[u] - depth[anc]);
    lift(v, depth[v] - depth[anc]);
    return res;
}
bool is_ancestor(int u, int v) { return lca(u, v) == u; }
};
/**

```

```

* Uso[1]: EulerTourTree<ll> et(t); et.
* subtree_range(u); et.subtree_size(u);
* Maapea el subarbol de cada nodo a un rango
* contiguo de la secuencia de entrada (tin/tout).
* Complejidad: O(1) por consulta.
*/
template <typename T = ll> struct EulerTourTree {
    vi in, out;
    EulerTourTree(const Tree<T>& tree) : in(tree.tin)
        , out(tree.tout) {}
    pair<int, int> subtree_range(int u) { return {in[
        u], out[u]}; }
    int subtree_size(int u) { return out[u] - in[u] +
        1; }
};

/**
* Uso[1]: HLD hld(n, true); hld.add_edge(u, v);
* hld.build(1); // 2o parametro: pesos en aristas
* vll values(n+1); values[hld.pos[u]] = weights[u
* ]; LazySegTree st(values);
* hld.process_path(u, v, [&](int l, int r) { st.
* add_range(l, r, val); });
* ll ans = 0; hld.process_path(u, v, [&](int l,
* int r) { ans += st.query(l, r); });
* Convierte caminos y subarboles en rangos
* continuos para inyectar estructuras 1D.
* El parametro values_on_edges del constructor
* indica si el peso esta en las aristas (true) o
* en los nodos (false); con aristas se ignora el
* LCA automaticamente.
* process_path: Llama a tu lambda particionando el
* camino en O(log N) rangos [l, r].
* process_subtree: Llama a tu lambda
* proporcionando el unico rango [l, r] del
* subarbol.
*/
struct HLD {
    const bool values_on_edges; // true si el peso
        esta en las aristas

    int n, timer;
    vector<vi> adj;
    vi parent, depth, sz, heavy, head, pos;

    HLD(int n, bool values_on_edges = false) :
        values_on_edges(values_on_edges), n(n), timer
        (0),
        adj(n + 1), parent(n + 1), depth(n + 1), sz(n +
        1), heavy(n + 1, -1), head(n + 1), pos(n + 1)
        {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    void dfs_sz(int u, int p, int d) {
        sz[u] = 1;
        parent[u] = p;
        depth[u] = d;
        heavy[u] = -1;
        int max_sub = 0;

        for (int v : adj[u]) {
            if (v != p) {
                dfs_sz(v, u, d + 1);

```

```

                sz[u] += sz[v];
                if (sz[v] > max_sub) {
                    max_sub = sz[v];
                    heavy[u] = v;
                }
            }
        }
    }

    void decompose(int u, int h) {
        head[u] = h;
        pos[u] = timer++;

        if (heavy[u] != -1) {
            decompose(heavy[u], h);
        }

        for (int v : adj[u]) {
            if (v != parent[u] && v != heavy[u]) {
                decompose(v, v);
            }
        }
    }

    void build(int root = 1) {
        timer = 0;
        dfs_sz(root, 0, 0);
        decompose(root, root);
    }

    template <class BinaryOperation>
    void process_path(int u, int v, BinaryOperation
        op) {
        while (head[u] != head[v]) {
            if (depth[head[u]] > depth[head[v]]) swap(u,
                v);
            op(pos[head[v]], pos[v]);
            v = parent[head[v]];
        }
        if (depth[u] > depth[v]) swap(u, v);
        if (values_on_edges && u == v) return;
        op(pos[u] + values_on_edges, pos[v]);
    }

    template <class BinaryOperation>
    void process_subtree(int u, BinaryOperation op) {
        op(pos[u] + values_on_edges, pos[u] + sz[u] -
            1);
    }
};

/**
* Uso[1]: CentroidPathProcessing cp(n); cp.K = 2;
* cp.add_edge(u, v); cp.solve(1); cout << cp.
* global.valid_pairs;
* Divide and Conquer con patron Query-then-Add.
* Problema resuelto por defecto: Conteo de caminos
* con exactamente K aristas.
*
* Modificables:
* - PathState: Estructura que almacena las
* propiedades de un camino desde el centroide
* actual.
* - GlobalState: Estructura que retiene los
* caminos de los subarboles ya procesados del
* centroide actual.
* - extend: Crea un nuevo PathState al atravesar

```

```

    una arista hacia el nodo 'v'.
* - query_and_merge: Combina el PathState actual
  con el GlobalState para contar respuestas
  validas.
* - add_to_global: Registra el PathState actual
  dentro del GlobalState.
* - clear_global: Revierte las adiciones iterando
  sobre los caminos para garantizar O(tamano
  subarbol).
*/
struct CentroidPathProcessing {
    struct PathState { ll dist; int edges; };

    struct GlobalState {
        vi freq_edges;
        ll valid_pairs = 0;
    } global;

    int K = 0;
    const PathState NEUTRAL_PATH = {0, 0};

    PathState extend(PathState p, int v, ll w) {
        return {p.dist + w, p.edges + 1};
    }

    void query_and_merge(PathState p) {
        if (p.edges <= K && K - p.edges <= n) { // K >
            n: no hay caminos validos
            global.valid_pairs += global.freq_edges[K - p
            .edges];
        }
    }

    void add_to_global(PathState p) {
        if (p.edges <= K) {
            global.freq_edges[p.edges]++;
        }
    }

    void clear_global(const vector<PathState>&
    all_paths) {
        for (const auto& p : all_paths) {
            if (p.edges <= K) {
                global.freq_edges[p.edges] = 0;
            }
        }
    }

    int n;
    vector<vector<pair<int, ll>>> adj;
    vi sz;
    vector<bool> removed;

    CentroidPathProcessing(int n) : n(n), adj(n + 1),
        sz(n + 1), removed(n + 1, false) {
        global.freq_edges.assign(n + 1, 0);
    }

    void add_edge(int u, int v, ll w = 1) {
        adj[u].pb({v, w});
        adj[v].pb({u, w});
    }

    void get_sizes(int u, int p) {
        sz[u] = 1;
        for (auto [v, w] : adj[u]) {
            if (v != p && !removed[v]) {
                get_sizes(v, u);
                sz[u] += sz[v];
            }
        }
    }

    int get_centroid(int u, int p, int total_size) {
        for (auto [v, w] : adj[u]) {
            if (v != p && !removed[v] && sz[v] >
                total_size / 2) {
                return get_centroid(v, u, total_size);
            }
        }
        return u;
    }

    void get_paths(int u, int p, PathState cur,
        vector<PathState>& paths) {
        paths.pb(cur);
        for (auto [v, w] : adj[u]) {
            if (v != p && !removed[v]) {
                get_paths(v, u, extend(cur, v, w), paths);
            }
        }
    }

    void solve(int u = 1) {
        get_sizes(u, 0);
        int c = get_centroid(u, 0, sz[u]);

        vector<PathState> all_paths_in_centroid;
        add_to_global(NEUTRAL_PATH);
        all_paths_in_centroid.pb(NEUTRAL_PATH);

        for (auto [v, w] : adj[c]) {
            if (!removed[v]) {
                vector<PathState> sub_paths;
                get_paths(v, c, extend(NEUTRAL_PATH, v, w),
                    sub_paths);

                for (auto& p : sub_paths) query_and_merge(p);
                for (auto& p : sub_paths) {
                    add_to_global(p);
                    all_paths_in_centroid.pb(p);
                }
            }
        }

        clear_global(all_paths_in_centroid);

        removed[c] = true;
        for (auto [v, w] : adj[c]) {
            if (!removed[v]) solve(v);
        }
    };
};

/**
* Uso[1]: CentroidTree<ll> ct(n); ct.add_edge(u, v
); ct.build(1);
* auto dist_lca = [&](int a, int b) { return lca.
dist_weighted(a, b); };
* ct.update(u, dist_lca); cout << ct.query(v,
dist_lca);
* Arbol topologico explicito inyectando la funcion

```

```

    de distancia.
* Problema resuelto por defecto: Consulta de la
  distancia mínima al nodo rojo mas cercano.
*
* Modificables:
* - State: Tipo de dato o estructura almacenada en
  cada centroide (ej. ll, multiset).
* - NEUTRAL: Valor que representa la ausencia de
  datos o el peor caso.
* - merge: Como se acumulan dos estados al
  actualizar o consultar (por defecto min).
* - combine: Como se une la distancia al centroide
  con el dato guardado para formar el candidato
  de respuesta (por defecto distancia + dato).
*/
template <typename State = ll>
struct CentroidTree {
    const State NEUTRAL = LINF;

    State merge(State a, State b) {
        return min(a, b);
    }

    State combine(State dist, State data) {
        return dist + data; // camino u -> centroide ->
                             dato
    }

    int n;
    vector<vi> adj;
    vi sz, c_parent;
    vector<bool> removed;
    vector<State> node_data;

    CentroidTree(int n) : n(n), adj(n + 1), sz(n + 1),
        c_parent(n + 1, 0),
        removed(n + 1, false),
        node_data(n + 1, NEUTRAL) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    void get_sizes(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) {
            if (v != p && !removed[v]) {
                get_sizes(v, u);
                sz[u] += sz[v];
            }
        }
    }

    int get_centroid(int u, int p, int total_size) {
        for (int v : adj[u]) {
            if (v != p && !removed[v] && sz[v] >
                total_size / 2) {
                return get_centroid(v, u, total_size);
            }
        }
        return u;
    }

    void build(int u = 1, int p = 0) {
        get_sizes(u, 0);
        int c = get_centroid(u, 0, sz[u]);
        c_parent[c] = p;

```

```

        removed[c] = true;
        for (int v : adj[c]) {
            if (!removed[v]) build(v, c);
        }
    }

    template<typename DistFunc>
    void update(int u, DistFunc dist_func) {
        for (int cur = u; cur; cur = c_parent[cur]) {
            State d = dist_func(u, cur);
            node_data[cur] = merge(node_data[cur], d);
        }
    }

    template<typename DistFunc>
    State query(int u, DistFunc dist_func) {
        State ans = NEUTRAL;
        for (int cur = u; cur; cur = c_parent[cur]) {
            State d = dist_func(u, cur);
            ans = merge(ans, combine(d, node_data[cur]));
        }
        return ans;
    }

    /**
    * Uso[1]: CentroidBinarySearch cbs(n); cbs.
    add_edge(u, v); int hidden_node = cbs.solve(1);
    * Busca un unico nodo reduciendo el arbol a la
    mitad en cada paso.
    * Problema resuelto por defecto: Encontrar un nodo
    oculto interactivamente preguntando distancias
    .
    *
    * Modificables:
    * - EvalState: La informacion recopilada de un
    nodo al evaluarlo.
    * - evaluate: Contiene la logica para consultar al
    juez o procesar el costo del nodo.
    * - get_direction: Oraculo que compara el estado
    del centroide actual con sus adyacentes
    para determinar el vecino que contiene la
    solucion, o retorna el centroide si es la
    respuesta.
    */
    struct CentroidBinarySearch {
        struct EvalState {
            int dist;
        };

        EvalState evaluate(int u) {
            cout << "? " << u << endl;
            int d;
            cin >> d;
            return {d};
        }

        int get_direction(int c) {
            EvalState current_state = evaluate(c);

            if (current_state.dist == 0) return c;

            int best_nxt = c;

            for (int v : adj[c]) {
                if (!removed[v]) {
                    EvalState neighbor_state = evaluate(v);

```

```

        if (neighbor_state.dist < current_state.
dist) {
            current_state = neighbor_state;
            best_nxt = v;
        }
    }
    return best_nxt;
}

int n;
vector<vi> adj;
vi sz;
vector<bool> removed;

CentroidBinarySearch(int n) : n(n), adj(n + 1),
    sz(n + 1), removed(n + 1, false) {}

void add_edge(int u, int v) {
    adj[u].pb(v);
    adj[v].pb(u);
}

void get_sizes(int u, int p) {
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v != p && !removed[v]) {
            get_sizes(v, u);
            sz[u] += sz[v];
        }
    }
}

int get_centroid(int u, int p, int total_size) {
    for (int v : adj[u]) {
        if (v != p && !removed[v] && sz[v] >
total_size / 2) {
            return get_centroid(v, u, total_size);
        }
    }
    return u;
}

int solve(int u = 1) {
    get_sizes(u, 0);
    int c = get_centroid(u, 0, sz[u]);

    int nxt = get_direction(c);
    if (nxt == c) return c;

    removed[c] = true;
    return solve(nxt);
}

/**
 * Uso[1]: DSUOnTree s(n); s.add_edge(u, v); s.
    set_colors(color_index); s.solve(root); s.ans[
    u];
 * Small-to-large (sack) para responder
    estadísticas de colores en cada subarbol.
 * Complejidad:  $O(N \log N)$ .
 */
struct DSUOnTree {
    int n, timer, distinct;
    vector<vi> adj;
    vi sz, heavy, tin, tout, euler_node, color, ans,

```

```

    cnt;
    DSUOnTree(int n) : n(n), timer(0), distinct(0),
        adj(n + 1), sz(n + 1), heavy(n + 1, -1),
        tin(n + 1), tout(n + 1), euler_node(n + 1),
        color(n + 1), ans(n + 1), cnt(n + 1) {}
    void add_edge(int u, int v) { adj[u].pb(v); adj[v]
        .pb(u); }
    void set_colors(const vi& c) {
        for (int i = 1; i <= n; i++) color[i] = c[i];
    }
    void dfs_size(int u, int p = 0) {
        sz[u] = 1;
        tin[u] = ++timer;
        euler_node[timer] = u;
        int max_sz = 0;
        for (int v : adj[u]) if (v != p) {
            dfs_size(v, u);
            sz[u] += sz[v];
            if (sz[v] > max_sz) max_sz = sz[v], heavy[u]
                = v;
        }
        tout[u] = timer;
    }
    void add_node(int u) { if (!cnt[color[u]]++)
        distinct++; }
    void remove_node(int u) { if (--cnt[color[u]])
        distinct--; }
    void dfs_sack(int u, int p, bool keep) {
        for (int v : adj[u]) if (v != p && v != heavy[u]
        ) dfs_sack(v, u, false);
        if (heavy[u] != -1) dfs_sack(heavy[u], u, true)
        ;
        add_node(u);
        for (int v : adj[u]) if (v != p && v != heavy[u]
        ) {
            for (int t = tin[v]; t <= tout[v]; t++)
                add_node(euler_node[t]);
            ans[u] = distinct;
            if (!keep) for (int t = tin[u]; t <= tout[u]; t
                ++) remove_node(euler_node[t]);
        }
    }
    void solve(int root = 1) { timer = 0; dfs_size(
        root); dfs_sack(root, 0, false); }
};

/**
 * Uso[1]: VirtualTree vt(n);
 * auto [root, nodes] = vt.build(
    query_nodes,
 * [&](int u, int v) { return lca.lca(u
    , v); },
 * [&](int u, int v) { return lca.
    is_ancestor(u, v); },
 * [&](int u, int v) { return lca.
    dist_weighted(u, v); },
 * [&](int a, int b) { return tree.tin[
    a] < tree.tin[b]; });
 * // Hacer DP usando vt.virt_adj[u]
    iterando sobre 'nodes'
 * Construye un arbol comprimido sobre un
    subconjunto de nodos manteniendo la topologia.
 * Recibe las funciones de lca, ancestro, distancia
    y orden de tin como lambdas para adaptarse a
    cualquier implementacion de LCA.
 * Complejidad: preproceso  $O(N)$ , construccion

```

```

    O(K log K) por query.
    */
    struct VirtualTree {
        int n;
        vector<vector<pair<int, ll>>> virt_adj;

        VirtualTree(int n) : n(n), virt_adj(n + 1) {}

        template <class LcaFunc, class IsAncFunc, class
            DistFunc, class TinCmp>
        pair<int, vi> build(vi nodes, LcaFunc get_lca,
            IsAncFunc is_anc, DistFunc get_dist, TinCmp
            tin_cmp) {
            if (nodes.empty()) return {0, {}};

            sort(all(nodes), tin_cmp);
            int k = nodes.size();
            for (int i = 0; i + 1 < k; i++) {
                nodes.pb(get_lca(nodes[i], nodes[i + 1]));
            }

            sort(all(nodes), tin_cmp);
            nodes.erase(unique(all(nodes)), nodes.end());

            for (int u : nodes) {
                virt_adj[u].clear();
            }

            vi stk = {nodes[0]};
            for (int i = 1; i < (int)nodes.size(); i++) {
                int u = nodes[i];
                while (!stk.empty() && !is_anc(stk.back(), u))
                {
                    stk.pop_back();
                }
                if (!stk.empty()) {
                    int anc = stk.back();
                    ll w = get_dist(anc, u);
                    virt_adj[anc].pb({u, w});
                    virt_adj[u].pb({anc, w});
                }
                stk.pb(u);
            }

            return {nodes[0], nodes};
        }
    };

    /**
    * Uso[1]: TreeHashing<ll>::are_isomorphic(t1, t2);
    * Determina si dos arboles son isomorfos (sin
    * considerar etiquetas) mediante hashing y los
    * centroides del struct Tree (reusa get_centers/
    * get_diameter).
    * Indexación: nodos 1-indexados.
    * Complejidad: O(N).
    */
    template <typename T = ll> struct TreeHashing {
        static uint64_t splitmix64(uint64_t x) {
            x += 0x9e3779b97f4a7c15;
            x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
            x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
            return x ^ (x >> 31);
        }

        static uint64_t hash_node(uint64_t x) { return
            splitmix64(x + 0x517cc1b727220a95); }

        static uint64_t rooted_hash(int u, int p, const
            Tree<T>& t) {
            vector<uint64_t> child;
            for (auto& e : t.adj[u]) if (e.to != p) child.
                pb(rooted_hash(e.to, u, t));
            sort(all(child));
            uint64_t h = 1;
            for (uint64_t c : child) h += hash_node(c);
            return splitmix64(h);
        }

        static bool are_isomorphic(const Tree<T>& t1,
            const Tree<T>& t2) {
            if (t1.n != t2.n) return false;
            auto c1 = t1.get_centers(), c2 = t2.get_centers
                ();
            for (int r1 : c1)
                for (int r2 : c2)
                    if (rooted_hash(r1, 0, t1) == rooted_hash(
                        r2, 0, t2)) return true;
            return false;
        }
    };

    /**
    * Uso[1]: RerootingDP r(n); r.add_edge(u, v); auto
    * ans = r.solve(root);
    * DP con rerooting para calcular la solucion para
    * todas las posibles raices; ajustar merge,
    * extend y finalize.
    * Complejidad: O(N).
    */
    struct RerootingDP {
        struct State { ll val; };
        int n;
        vector<vi> adj;
        vector<State> dp_down, ans;
        const State NEUTRAL = {0};
        State merge(State a, State b) { return {max(a.val
            , b.val)}; }
        State extend(State s, int v, int u) { return {s.
            val + 1}; } // transicion por arista
        State finalize(State s, int u) { return s; }
        RerootingDP(int n) : n(n), adj(n + 1), dp_down(n
            + 1, NEUTRAL), ans(n + 1, NEUTRAL) {}
        void add_edge(int u, int v) { adj[u].pb(v); adj[v
            ].pb(u); }
        void dfs_down(int u, int p) {
            State cur = NEUTRAL;
            for (int v : adj[u]) if (v != p) dfs_down(v, u)
                , cur = merge(cur, extend(dp_down[v], v, u));
            dp_down[u] = finalize(cur, u);
        }
        void dfs_reroot(int u, int p, State from_parent)
            {
                vi children;
                for (int v : adj[u]) if (v != p) children.pb(v)
                    ;
                int k = children.size();
                vector<State> pref(k + 1, NEUTRAL), suff(k + 1,
                    NEUTRAL);
                for (int i = 0; i < k; i++) pref[i + 1] = merge
                    (pref[i], extend(dp_down[children[i]], children
                        [i], u));
                for (int i = k - 1; i >= 0; i--) suff[i] =
                    merge(suff[i + 1], extend(dp_down[children[i]],
                        children[i], u));
            }
    };

```

```

    ans[u] = finalize(merge(pref[k], from_parent),
u);
    for (int i = 0; i < k; i++) {
        State outside = merge(merge(pref[i], suff[i +
1]), from_parent);
        dfs_reroot(children[i], u, extend(finish(
outside, u), u, children[i]));
    }
}
vector<State> solve(int root = 1) {
    dfs_down(root, 0);
    dfs_reroot(root, 0, NEUTRAL);
    return ans;
}
};

/**
 * Uso[1]: FastLCA<ll> fl(t); fl.lca(u, v); fl.
dist_edges(u, v);
 * LCA por Euler + RMQ con respuesta en tiempo
constante por consulta.
 * Complejidad: preproceso  $O(N \log N)$ , consulta  $O(1)$ .
 */
template <typename T = ll> struct FastLCA {
    const Tree<T>& tree;
    vi tin, euler, euler_depth, log_table;
    vector<vi> st;
    FastLCA(const Tree<T>& tree) : tree(tree), tin(
tree.n + 1) {
        auto dfs = [&](auto&& self, int u, int p, int d
) -> void {
            tin[u] = euler.size();
            euler.pb(u);
            euler_depth.pb(d);
            for (auto& e : tree.adj[u]) if (e.to != p) {
                self(self, e.to, u, d + 1);
                euler.pb(u);
                euler_depth.pb(d);
            }
        };
        dfs(dfs, tree.root, 0, 0);
        int m = euler.size();
        log_table.assign(m + 1, 0);
        for (int i = 2; i <= m; i++) log_table[i] =
log_table[i / 2] + 1;
        int K = log_table[m] + 1;
        st.assign(K, vi(m));
        for (int i = 0; i < m; i++) st[0][i] = i;
        for (int i = 1; i < K; i++)
            for (int j = 0; j + (1 <= i) <= m; j++) {
                int l = st[i - 1][j], r = st[i - 1][j + (1
<= i - 1)];
                st[i][j] = euler_depth[l] < euler_depth[r]
? l : r;
            }
    }
    int lca(int u, int v) {
        int l = tin[u], r = tin[v];
        if (l > r) swap(l, r);
        int i = log_table[r - l + 1];
        int a = st[i][l], b = st[i][r - (1 <= i) + 1];
        return euler_depth[a] < euler_depth[b] ? euler[
a] : euler[b];
    }
    int dist_edges(int u, int v) { return tree.depth[
u] + tree.depth[v] - 2 * tree.depth[lca(u, v)];
}
};

/**
 * Uso[1]: MoOnTrees<ll> mo(n); mo.dfs_euler(root,
0, t); auto ans = mo.solve(queries, fast_lca);
 * Algoritmo de Mo sobre caminos de un arbol; se
implementa toggle(u) (frecuencia de colores,
suma de valores, etc.) y se leen ans[i] por
query.
 * Una query real es un par de nodos (u, v); por
ejemplo, para contar colores distintos en el
camino u-v se guarda cnt[color] y current_ans =
numero de colores con cnt > 0.
 * Complejidad:  $O((N + Q)\sqrt{N})$  toggle esperados.
 */
template <typename T = ll> struct MoOnTrees {
    struct Query {
        int l, r, lca, id, block_size;
        bool operator<(const Query& o) const {
            int b1 = l / block_size, b2 = o.l /
block_size;
            if (b1 != b2) return b1 < b2;
            return (b1 & 1) ? (r < o.r) : (r > o.r);
        }
    };
    int n, timer;
    vi tin, tout, euler;
    vector<bool> in_path;
    ll current_ans = 0;
    MoOnTrees(int n) : n(n), timer(0), tin(n + 1),
tout(n + 1), euler(2 * n + 1), in_path(n + 1,
false) {}
    void dfs_euler(int u, int p, const Tree<T>& tree)
    {
        tin[u] = ++timer;
        euler[timer] = u;
        for (auto& e : tree.adj[u]) if (e.to != p)
            dfs_euler(e.to, u, tree);
        tout[u] = ++timer;
        euler[timer] = u;
    }
    void toggle(int u) {
        // TODO: actualizar respuesta al agregar/
        // remover el nodo u (p.ej. frecuencias de colores
        )
        in_path[u] = !in_path[u];
    }
    vll solve(vector<pair<int, int>>& raw_queries,
const FastLCA<T>& fast_lca) {
        int q = raw_queries.size();
        int bs = max(1, (int)(2 * n / sqrt(max(1, q))))
;
        vector<Query> queries;
        for (int i = 0; i < q; i++) {
            auto [u, v] = raw_queries[i];
            if (tin[u] > tin[v]) swap(u, v);
            int anc = fast_lca.lca(u, v);
            queries.pb(anc == u ? Query{tin[u], tin[v],
0, i, bs} : Query{tout[u], tin[v], anc, i, bs})
;
        }
        sort(all(queries));
        vll ans(q);
        int curr_l = 1, curr_r = 0;

```

```

for (auto& mq : queries) {
    while (curr_l > mq.l) toggle(euler[--curr_l]);
    while (curr_r < mq.r) toggle(euler[++curr_r]);
    while (curr_l < mq.l) toggle(euler[curr_l++]);
    while (curr_r > mq.r) toggle(euler[curr_r--]);
    if (mq.lca) toggle(mq.lca);
    ans[mq.id] = current_ans;
    if (mq.lca) toggle(mq.lca);
}
return ans;
};

```

Range Queries

```

/**
 * Uso[0]: SparseTable st(a); st.query(L, R);
 * Consultas estaticas en rangos para operaciones
 * idempotentes (min, max, gcd).
 * Complejidad: construccion  $O(N \log N)$ , consulta
 *  $O(1)$ .
 */
struct SparseTable {
    int n;
    vector<vll> st;
    SparseTable(const vll& a) : n(a.size()) {
        int log_n = log2(n) + 1;
        st.assign(n, vll(log_n));
        for (int i = 0; i < n; i++) st[i][0] = a[i];
        for (int j = 1; j < log_n; j++)
            for (int i = 0; i + (1 < j) <= n; i++)
                st[i][j] = min(st[i][j - 1], st[i + (1 < j) - 1][j - 1]); // min/max/gcd
    }
    ll query(int L, int R) {
        int j = log2(R - L + 1);
        return min(st[L][j], st[R - (1 < j) + 1][j]);
    }
};

/**
 * Uso[1]: FenwickTree ft(n); ft.add(i, x); ll s =
 * ft.query(2, 3);
 * Arbol de Fenwick para sumas de prefijos y rangos
 * con actualizaciones puntuales.
 * Complejidad: actualizacion y consulta  $O(\log N)$ .
 */
struct FenwickTree {
    int n;
    vll bit;
    FenwickTree(int n) : n(n), bit(n + 1, 0) {}
    void add(int idx, ll val) { for (; idx <= n; idx
        += idx & -idx) bit[idx] += val; }
    ll sum(int idx) {
        ll res = 0;
        for (; idx > 0; idx -= idx & -idx) res += bit[
            idx];
        return res;
    }
    ll query(int L, int R) { return sum(R) - sum(L -
        1); }
};

```

```

* Uso[0]: IterativeSegTree st(a); st.update(pos,
    val); st.query(L, R);
* Arbol de segmentos iterativo para suma (o min/
    max) con actualizacion puntual.
* Complejidad: actualizacion y consulta  $O(\log N)$ .
*/
struct IterativeSegTree {
    int n;
    vll st;
    IterativeSegTree(const vll& a) : n(a.size()), st
        (2 * a.size(), 0) {
        for (int i = 0; i < n; i++) st[n + i] = a[i];
        for (int i = n - 1; i > 0; i--) st[i] = st[i <
            1] + st[i < 1 | 1]; // + / min / max
    }
    void update(int pos, ll val) {
        for (st[pos += n] = val; pos > 1; pos >>= 1) st
            [pos >> 1] = st[pos] + st[pos ^ 1];
    }
    ll query(int L, int R) {
        ll resL = 0, resR = 0;
        for (L += n, R += n + 1; L < R; L >>= 1, R >>=
            1) {
            if (L & 1) resL += st[L++];
            if (R & 1) resR += st[--R];
        }
        return resL + resR;
    }
};

/**
 * Uso[0]: RecursiveSegTree st(a); st.update(pos,
    val); st.query(L, R); st.find_first(val);
* Arbol de segmentos recursivo con estructura de
    nodo y metodo merge para hacer variaciones
    faciles (max, +, min).
* Para cambiar la operacion, edita merge (mezcla
    de dos nodos) y neutral (identidad de la
    operacion).
* Indexación: 0-indexado.
* Complejidad: actualizacion y consulta  $O(\log N)$ .
*/
struct RecursiveSegTree {
    struct Node { ll val; };
    int n;
    vector<Node> st;
    Node merge(const Node& a, const Node& b) { return
        {max(a.val, b.val)}; } // max / + / min
    Node neutral() { return {-LINF}; } // identidad
        de merge: -LINF (max), 0 (suma), LINF (min)
    RecursiveSegTree(const vll& a) : n(a.size()), st
        (4 * a.size()) { build(1, 0, n - 1, a); }
    void build(int p, int L, int R, const vll& a) {
        if (L == R) { st[p] = {a[L]}; return; }
        int mid = (L + R) / 2;
        build(p << 1, L, mid, a);
        build(p << 1 | 1, mid + 1, R, a);
        st[p] = merge(st[p << 1], st[p << 1 | 1]);
    }
    void update(int p, int L, int R, int pos, ll val)
        {
        if (L == R) { st[p] = {val}; return; } // o st[
            p].val += val para sumar
        int mid = (L + R) / 2;
        pos <= mid ? update(p << 1, L, mid, pos, val) :
            update(p << 1 | 1, mid + 1, R, pos, val);
    }
};

```

```

    st[p] = merge(st[p <= 1], st[p <= 1 | 1]);
}
Node query(int p, int L, int R, int qL, int qR) {
    if (qL > R || qR < L) return neutral();
    if (qL <= L && R <= qR) return st[p];
    int mid = (L + R) / 2;
    return merge(query(p <= 1, L, mid, qL, qR),
        query(p <= 1 | 1, mid + 1, R, qL, qR));
}
// Primer indice con valor >= val (asume arbol de
// max). -1 si no existe
int find_first(int p, int L, int R, ll val) {
    if (st[p].val < val) return -1;
    if (L == R) return L;
    int mid = (L + R) / 2;
    return st[p <= 1].val >= val ? find_first(p <=
        1, L, mid, val) : find_first(p <= 1 | 1, mid +
        1, R, val);
}
void update(int pos, ll val) { update(1, 0, n -
    1, pos, val); }
ll query(int L, int R) { return query(1, 0, n -
    1, L, R).val; }
int find_first(ll val) { return find_first(1, 0,
    n - 1, val); }
};

/**
 * Uso[0]: LazySegTree st(a); st.add_range(L, R, v)
 * ; st.set_range(L, R, v); st.query(L, R); st.get
 * (pos);
 * Arbol de segmentos con propagacion perezosa y
 * estructura de nodo + metodo merge para suma (o
 * min/max) con add y set por rango.
 * Para cambiar la operacion, edita merge y la
 * manera en que apply_add/apply_set combinan el
 * valor de cada nodo.
 * Indexación: 0-indexado.
 * Complejidad: actualizacion y consulta de rango
 * O(log N).
 */
struct LazySegTree {
    struct Node { ll sum = 0; };
    int n;
    vector<Node> st;
    vll lazy_add, lazy_set;
    vector<bool> marked;
    Node merge(const Node& a, const Node& b) { return
        {a.sum + b.sum}; } // + / min / max
    LazySegTree(const vll& a) : n(a.size()), st(4 * a
        .size()), lazy_add(4 * a.size()),
        lazy_set(4 * a.size()), marked(4 * a.size(),
        false) { build(1, 0, n - 1, a); }
    void apply_set(int p, ll val, int L, int R) {
        st[p].sum = val * (R - L + 1); // para min/max:
        st[p].sum = val
        lazy_set[p] = val;
        lazy_add[p] = 0;
        marked[p] = true;
    }
    void apply_add(int p, ll val, int L, int R) {
        st[p].sum += val * (R - L + 1); // para min/max
        : st[p].sum += val
        if (marked[p]) lazy_set[p] += val;
        else lazy_add[p] += val;
    }
};

```

```

void push(int p, int L, int R) {
    int mid = (L + R) / 2;
    if (marked[p]) {
        apply_set(p <= 1, lazy_set[p], L, mid);
        apply_set(p <= 1 | 1, lazy_set[p], mid + 1, R
        );
        marked[p] = false;
    }
    if (lazy_add[p]) {
        apply_add(p <= 1, lazy_add[p], L, mid);
        apply_add(p <= 1 | 1, lazy_add[p], mid + 1, R
        );
        lazy_add[p] = 0;
    }
}
void build(int p, int L, int R, const vll& a) {
    if (L == R) { st[p] = {a[L]}; return; }
    int mid = (L + R) / 2;
    build(p <= 1, L, mid, a);
    build(p <= 1 | 1, mid + 1, R, a);
    st[p] = merge(st[p <= 1], st[p <= 1 | 1]);
}
void update(int p, int L, int R, int qL, int qR,
    ll val, int type) { // 1 = add, 2 = set
    if (qL > R || qR < L) return;
    if (qL <= L && R <= qR) { type == 1 ? apply_add
        (p, val, L, R) : apply_set(p, val, L, R);
        return; }
    push(p, L, R);
    int mid = (L + R) / 2;
    update(p <= 1, L, mid, qL, qR, val, type);
    update(p <= 1 | 1, mid + 1, R, qL, qR, val,
        type);
    st[p] = merge(st[p <= 1], st[p <= 1 | 1]);
}
Node query(int p, int L, int R, int qL, int qR) {
    if (qL > R || qR < L) return {0};
    if (qL <= L && R <= qR) return st[p];
    push(p, L, R);
    int mid = (L + R) / 2;
    return merge(query(p <= 1, L, mid, qL, qR),
        query(p <= 1 | 1, mid + 1, R, qL, qR));
}
Node get_point(int p, int L, int R, int pos) {
    if (L == R) return st[p];
    push(p, L, R);
    int mid = (L + R) / 2;
    return pos <= mid ? get_point(p <= 1, L, mid,
        pos) : get_point(p <= 1 | 1, mid + 1, R, pos);
}
void add_range(int L, int R, ll val) { update(1,
    0, n - 1, L, R, val, 1); }
void set_range(int L, int R, ll val) { update(1,
    0, n - 1, L, R, val, 2); }
ll query(int L, int R) { return query(1, 0, n -
    1, L, R).sum; }
ll get(int pos) { return get_point(1, 0, n - 1,
    pos).sum; }
};

/**
 * Uso[1]: DynamicSegTree st(max_rango); st.update(
 * pos, val); st.query(L, R);
 * Arbol de segmentos dinamico (sparse) para

```

```

    indices grandes sin compresion previa.
    * Complejidad: actualizacion y consulta
       $O(\log \max\_rango)$ .
    */
    struct DynamicSegTree {
        struct Node { ll sum = 0; int left = 0, right =
            0; };
        vector<Node> st;
        ll max_range;
        int root;
        DynamicSegTree(ll max_range = 1e9) : max_range(
            max_range), root(0) { st.emplace_back(); }
        int new_node() { st.emplace_back(); return st.
            size() - 1; }
        int update(int node, ll L, ll R, ll pos, ll val)
        {
            if (!node) node = new_node();
            st[node].sum += val;
            if (L == R) return node;
            ll mid = (L + R) / 2;
            if (pos <= mid) st[node].left = update(st[node]
                .left, L, mid, pos, val);
            else st[node].right = update(st[node].right,
                mid + 1, R, pos, val);
            return node;
        }
        ll query(int node, ll L, ll R, ll qL, ll qR) {
            if (!node || qL > R || qR < L) return 0;
            if (qL <= L && R <= qR) return st[node].sum;
            ll mid = (L + R) / 2;
            return query(st[node].left, L, mid, qL, qR) +
                query(st[node].right, mid + 1, R, qL, qR);
        }
        void update(ll pos, ll val) { root = update(root,
            1, max_range, pos, val); }
        ll query(ll L, ll R) { return query(root, 1,
            max_range, L, R); }
    };

    /**
    * Uso[0]: PersistentSegTree pst(a); int v = pst.
      update(version, pos, val); pst.query(v, L, R);
    * Arbol de segmentos persistente con versiones
      historicas y consultas sobre cualquier version.
    * Complejidad: actualizacion y consulta  $O(\log N)$ ;
      espacio  $O(N \log N)$ .
    */
    struct PersistentSegTree {
        struct Node { ll sum; int left, right; Node(ll s
            = 0, int l = 0, int r = 0) : sum(s), left(l),
            right(r) {} };
        vector<Node> st;
        vi roots;
        int n;
        int new_node(ll sum, int left, int right) { st.
            emplace_back(sum, left, right); return st.size
            () - 1; }
        int build(int L, int R, const vll& a) {
            if (L == R) return new_node(a[L], 0, 0);
            int mid = (L + R) / 2;
            int l = build(L, mid, a), r = build(mid + 1, R,
                a);
            return new_node(st[l].sum + st[r].sum, l, r);
        }
        int update(int old, int L, int R, int pos, ll val
            ) {

```

```

            if (L == R) return new_node(st[old].sum + val,
                0, 0);
            int mid = (L + R) / 2;
            int l = st[old].left, r = st[old].right;
            if (pos <= mid) l = update(l, L, mid, pos, val)
                ;
            else r = update(r, mid + 1, R, pos, val);
            return new_node(st[l].sum + st[r].sum, l, r);
        }
        ll query(int node, int L, int R, int qL, int qR)
        {
            if (!node || qL > R || qR < L) return 0;
            if (qL <= L && R <= qR) return st[node].sum;
            int mid = (L + R) / 2;
            return query(st[node].left, L, mid, qL, qR) +
                query(st[node].right, mid + 1, R, qL, qR);
        }
        PersistentSegTree(const vll& a) : n(a.size()) {
            st.emplace_back();
            roots.pb(build(0, n - 1, a));
        }
        int update(int prev_version, int pos, ll val) {
            // crea nueva version
            roots.pb(update(roots[prev_version], 0, n - 1,
                pos, val));
            return roots.size() - 1;
        }
        ll query(int version, int L, int R) { return
            query(roots[version], 0, n - 1, L, R); }
    };

    /**
    * Uso[1]: FenwickTree2D ft(n, m); ft.add(r, c, v);
      ft.query(r1, c1, r2, c2);
    * Arbol de Fenwick bidimensional para sumas de
      submatrices con actualizaciones puntuales.
    * Complejidad: actualizacion y consulta
       $O(\log n \cdot \log m)$ .
    */
    struct FenwickTree2D {
        int n, m;
        vector<vll> bit;
        FenwickTree2D(int n, int m) : n(n), m(m), bit(n +
            1, vll(m + 1, 0)) {}
        void add(int r, int c, ll val) {
            for (int i = r; i <= n; i += i & -i)
                for (int j = c; j <= m; j += j & -j) bit[i][j
                    ] += val;
        }
        ll sum(int r, int c) {
            ll res = 0;
            for (int i = r; i > 0; i -= i & -i)
                for (int j = c; j > 0; j -= j & -j) res +=
                    bit[i][j];
            return res;
        }
        ll query(int r1, int c1, int r2, int c2) {
            return sum(r2, c2) - sum(r1 - 1, c2) - sum(r2,
                c1 - 1) + sum(r1 - 1, c1 - 1);
        }
    };

    /**
    * Uso[0]: MergeSortTree st(a); st.count_less_than(
      L, R, val); st.count_less_equal(L, R, val); st.
      update(pos, val);

```

* *Arbol de segmentos con listas ordenadas para contar elementos menores o iguales a un valor en un rango; soporta actualizaciones puntuales.*
 * *Indexación: 0-indexado.*
 * *Complejidad: construccion $O(N \log N)$, consulta $O(\log^2 N)$, actualización $O(N)$ (reconstruye el camino por mezcla).*

```

*/
struct MergeSortTree {
    int n;
    vector<vll> st;
    MergeSortTree(const vll& a) : n(a.size()), st(4 *
        a.size()) { build(1, 0, n - 1, a); }
    void build(int p, int L, int R, const vll& a) {
        if (L == R) { st[p] = {a[L]}; return; }
        int mid = (L + R) / 2;
        build(p << 1, L, mid, a);
        build(p << 1 | 1, mid + 1, R, a);
        merge_children(p);
    }
    void merge_children(int p) {
        st[p].resize(st[p << 1].size() + st[p << 1 | 1].size());
        merge(all(st[p << 1]), all(st[p << 1 | 1]), st[p].begin());
    }
    void update(int p, int L, int R, int pos, ll val)
    {
        if (L == R) { st[p] = {val}; return; }
        int mid = (L + R) / 2;
        pos <= mid ? update(p << 1, L, mid, pos, val) :
            update(p << 1 | 1, mid + 1, R, pos, val);
        merge_children(p);
    }
    int less_than(int p, int L, int R, int qL, int qR,
        ll val) {
        if (qL > R || qR < L) return 0;
        if (qL <= L && R <= qR) return lower_bound(all(
            st[p]), val) - st[p].begin();
        int mid = (L + R) / 2;
        return less_than(p << 1, L, mid, qL, qR, val) +
            less_than(p << 1 | 1, mid + 1, R, qL, qR, val);
    }
    int less_equal(int p, int L, int R, int qL, int
        qR, ll val) {
        if (qL > R || qR < L) return 0;
        if (qL <= L && R <= qR) return upper_bound(all(
            st[p]), val) - st[p].begin();
        int mid = (L + R) / 2;
        return less_equal(p << 1, L, mid, qL, qR, val)
            + less_equal(p << 1 | 1, mid + 1, R, qL, qR,
                val);
    }
    void update(int pos, ll val) { update(1, 0, n -
        1, pos, val); }
    int count_less_than(int L, int R, ll val) {
        return less_than(1, 0, n - 1, L, R, val); }
    int count_less_equal(int L, int R, ll val) {
        return less_equal(1, 0, n - 1, L, R, val); }
};

```

/**

* *Uso[0]: SqrtDecomposition sq(a); sq.update(pos, val); sq.query(L, R);*

* *Descomposicion en bloques para sumas de rangos con actualizaciones puntuales.*

* *Complejidad: actualización y consulta $O(\sqrt{N})$.*

*/

```

struct SqrtDecomposition {
    int n, block_size;
    vll a, blocks;
    SqrtDecomposition(const vll& arr) : n(arr.size())
        , block_size(max(1, (int)sqrt(n))), a(arr) {
        blocks.assign((n + block_size - 1) / block_size
            , 0);
        for (int i = 0; i < n; i++) blocks[i /
            block_size] += a[i];
    }
    void update(int pos, ll val) {
        int b = pos / block_size;
        a[pos] = val;
        blocks[b] = 0;
        int end = min(n, (b + 1) * block_size);
        for (int i = b * block_size; i < end; i++)
            blocks[b] += a[i];
    }
    ll query(int L, int R) {
        int bL = L / block_size, bR = R / block_size;
        ll res = 0;
        if (bL == bR) {
            for (int i = L; i <= R; i++) res += a[i];
            return res;
        }
        for (int i = L; i < (bL + 1) * block_size; i++)
            res += a[i];
        for (int b = bL + 1; b < bR; b++) res += blocks
            [b];
        for (int i = bR * block_size; i <= R; i++) res
            += a[i];
        return res;
    }
};

```

/**

* *Uso[0]: Treap t; t.insert(x); t.erase(x); t.kth_element(k); t.count_less_than(x);*
 * *Arbol binario de busqueda balanceado aleatoriamente (treap) con consultas por orden.*
 * *Complejidad: cada operacion $O(\log N)$ esperado.*
 */

```

mt19937_64 rng(chrono::steady_clock::now().
    time_since_epoch().count());
struct TreapNode {
    ll key, prior;
    int sz;
    TreapNode *l, *r;
    TreapNode(ll key) : key(key), prior(rng()), sz(1)
        , l(nullptr), r(nullptr) {}
};
typedef TreapNode* pNode;
struct Treap {
    pNode root = nullptr;
    int sz(pNode t) { return t ? t->sz : 0; }
    void upd(pNode t) { if (t) t->sz = 1 + sz(t->l) +
        sz(t->r); }
    void split(pNode t, ll key, pNode& l, pNode& r) {
        // l: keys <= key
        if (!t) { l = r = nullptr; return; }
        if (key >= t->key) split(t->r, key, t->r, r), l

```

```

    = t;
    else split(t->l, key, l, t->l), r = t;
    upd(t);
}
void merge(pNode& t, pNode l, pNode r) {
    if (!l || !r) { t = l ? l : r; return; }
    if (l->prior > r->prior) merge(l->r, l->r, r),
        t = l;
    else merge(r->l, l, r->l), t = r;
    upd(t);
}
void insert(ll key) {
    pNode l, r;
    split(root, key, l, r);
    pNode node = new TreapNode(key);
    merge(l, l, node);
    merge(root, l, r);
}
void erase(ll key) {
    pNode l, m, r;
    split(root, key - 1, l, r);
    split(r, key, m, r);
    if (m) {
        pNode tmp = m;
        merge(m, m->l, m->r);
        delete tmp;
    }
    merge(root, l, m);
    merge(root, root, r);
}
ll kth_element(int k) { // k-esimo menor (0-
    indexed). LINF si fuera de rango
    pNode cur = root;
    while (cur) {
        int ls = sz(cur->l);
        if (ls == k) return cur->key;
        if (ls > k) cur = cur->l;
        else k -= ls + 1, cur = cur->r;
    }
    return LINF;
}
int count_less_than(ll key) {
    pNode l, r;
    split(root, key - 1, l, r);
    int res = sz(l);
    merge(root, l, r);
    return res;
}
};

/**
 * Uso[0]: ImplicitTreap it; it.insert(pos, val);
 * it.erase(pos); it.query_range(L, R); it.
 * reverse_range(L, R);
 * Treap implicito para secuencias dinamicas con
 * insercion/borrado por posicion, suma de rangos
 * y reversiones perezosas.
 * Complejidad: cada operacion O(log N) esperado.
 */
struct ImplicitNode {
    ll val, sum, prior;
    int sz;
    bool lazy_rev;
    ImplicitNode *l, *r;
    ImplicitNode(ll val) : val(val), sum(val), prior(
        rng()), sz(1), lazy_rev(false), l(nullptr), r(
        nullptr) {}

```

```

};
typedef ImplicitNode* pINode;
struct ImplicitTreap {
    pINode root = nullptr;
    int sz(pINode t) { return t ? t->sz : 0; }
    ll sum(pINode t) { return t ? t->sum : 0; }
    void push(pINode t) {
        if (t && t->lazy_rev) {
            t->lazy_rev = false;
            swap(t->l, t->r);
            if (t->l) t->l->lazy_rev ^= true;
            if (t->r) t->r->lazy_rev ^= true;
        }
    }
    void upd(pINode t) {
        if (t) t->sz = 1 + sz(t->l) + sz(t->r), t->sum
            = t->val + sum(t->l) + sum(t->r);
    }
    void split(pINode t, int k, pINode& l, pINode& r)
        { // l: primeros k elementos
        if (!t) { l = r = nullptr; return; }
        push(t);
        int implicit = sz(t->l) + 1;
        if (k >= implicit) split(t->r, k - implicit, t
            ->r, r), l = t;
        else split(t->l, k, l, t->l), r = t;
        upd(t);
    }
    void merge(pINode& t, pINode l, pINode r) {
        push(l); push(r);
        if (!l || !r) { t = l ? l : r; return; }
        if (l->prior > r->prior) merge(l->r, l->r, r),
            t = l;
        else merge(r->l, l, r->l), t = r;
        upd(t);
    }
    void insert(int pos, ll val) {
        pNode l, r;
        split(root, pos, l, r);
        pNode node = new ImplicitNode(val);
        merge(l, l, node);
        merge(root, l, r);
    }
    void erase(int pos) {
        pNode l, mid, r;
        split(root, pos, l, r);
        split(r, 1, mid, r);
        delete mid;
        merge(root, l, r);
    }
    void reverse_range(int L, int R) {
        pNode l, mid, r;
        split(root, L, l, mid);
        split(mid, R - L + 1, mid, r);
        mid->lazy_rev ^= true;
        merge(root, l, mid);
        merge(root, root, r);
    }
    ll query_range(int L, int R) {
        pNode l, mid, r;
        split(root, L, l, mid);
        split(mid, R - L + 1, mid, r);
        ll ans = sum(mid);
        merge(root, l, mid);
        merge(root, root, r);
        return ans;
    }

```

```

    }
};

/**
 * Uso[0]: Mo mo(n); mo.add_query(1, r); auto ans =
 * mo.solve(add, remove, get_ans);
 * Algoritmo de Mo 1D para responder consultas de
 * rango offline cuando add y remove son O(1).
 * Ejemplo de los callbacks (contar distintos en [1
 * , r]): int cnt[MAXV] = {}, d = 0;
 * auto add = [&](int i){ if (!cnt[a[i]]++) d++;
 * };
 * auto remove = [&](int i){ if (--cnt[a[i]]) d
 * --; };
 * auto get = [&]() { return (11)d; };
 * Complejidad: O((N+Q)√N) movimientos.
 */
struct Mo {
    struct Query { int l, r, id, block; };
    int n, bs;
    vector<Query> qs;
    Mo(int n, int bs = 0) : n(n), bs(bs ? bs : max(1,
        (int)sqrt(n))) {}
    void add_query(int l, int r) { qs.pb({l, r, (int)
        qs.size(), 1 / bs}); }
    template <class Add, class Remove, class GetAns>
    vector<ll> solve(Add add, Remove remove, GetAns
        get_ans) {
        vector<ll> res(qs.size());
        sort(all(qs), [](const Query& a, const Query& b
        ) {
            if (a.block != b.block) return a.block < b.
            block;
            return (a.block & 1) ? a.r > b.r : a.r < b.r;
            // odd-even para acelerar
        });
        int cur_l = 0, cur_r = -1;
        for (const Query& q : qs) {
            while (cur_l > q.l) add(--cur_l);
            while (cur_r < q.r) add(++cur_r);
            while (cur_l < q.l) remove(cur_l++);
            while (cur_r > q.r) remove(cur_r--);
            res[q.id] = get_ans();
        }
        return res;
    }
};

/**
 * Uso[0]: MoWithUpdates mo(n); mo.add_update(pos,
 * antes, despues); mo.add_query(1, r); auto ans =
 * mo.solve(add, remove, apply, revert, get_ans);
 * Algoritmo de Mo con actualizaciones puntuales (
 * Mo 3D); responde consultas sobre el arreglo en
 * un instante dado.
 * apply(u, L, R) aplica el update u al rango
 * actual [L,R] y revert lo deshace; L y R son los
 * parametros que recibe el callback.
 * Ejemplo (distintos): si u.pos cae en [L,R] se
 * retira el valor viejo y se agrega el nuevo;
 * luego arr[u.pos] = u.new_val (y al reves en
 * revert).
 * Complejidad: O((N+Q)^(5/3)) movimientos esperados.
 */
struct MoWithUpdates {
    struct Query { int l, r, t, id; };

```

```

    struct Update { int pos; ll old_val, new_val; };
    int n, bs;
    vector<Query> qs;
    vector<Update> ups;
    MoWithUpdates(int n, int bs = 0) : n(n), bs(bs ?
        bs : max(1, (int)pow(n, 2.0 / 3.0))) {}
    void add_update(int pos, ll old_val, ll new_val)
    { ups.pb({pos, old_val, new_val}); }
    void add_query(int l, int r) { qs.pb({l, r, (int)
        ups.size(), (int)qs.size()}); }
    template <class Add, class Remove, class Apply,
        class Revert, class GetAns>
    vector<ll> solve(Add add, Remove remove, Apply
        apply, Revert revert, GetAns get_ans) {
        vector<ll> res(qs.size());
        sort(all(qs), [&](const Query& a, const Query&
        b) {
            int ba = a.l / bs, bb = b.l / bs;
            if (ba != bb) return ba < bb;
            int ca = a.r / bs, cb = b.r / bs;
            if (ca != cb) return ca < cb;
            return a.t < b.t;
        });
        int cur_l = 0, cur_r = -1, cur_t = 0;
        for (const Query& q : qs) {
            while (cur_t < q.t) { apply(ups[cur_t], cur_l
            , cur_r); cur_t++; }
            while (cur_t > q.t) { cur_t--; revert(ups[
            cur_t], cur_l, cur_r); }
            while (cur_l > q.l) add(--cur_l);
            while (cur_r < q.r) add(++cur_r);
            while (cur_l < q.l) remove(cur_l++);
            while (cur_r > q.r) remove(cur_r--);
            res[q.id] = get_ans();
        }
        return res;
    }
};

/**
 * Uso[0]: RollbackMo mo(n); mo.add_query(1, r); mo
 * .solve(add, current, snapshot, rollback, reset)
 * ;
 * Variante de Mo con rollback para estructuras
 * donde agregar es O(1) pero eliminar es costoso
 * o imposible.
 * Ejemplo (distintos): add(i) inserta a[i] y
 * guarda i en un stack; snapshot() limpia el
 * stack; rollback() deshace los adds del stack;
 * reset() vacia toda la estructura.
 * Complejidad: O((N+Q)√N) operaciones de add.
 */
struct RollbackMo {
    struct Query { int l, r, id; };
    int n, bs;
    vector<Query> qs;
    RollbackMo(int n, int bs = 0) : n(n), bs(bs ? bs
        : max(1, (int)sqrt(n))) {}
    void add_query(int l, int r) { qs.pb({l, r, (int)
        qs.size()}); }
    template <class Add, class Current, class
        Snapshot, class Rollback, class Reset>
    vector<ll> solve(Add add, Current current,
        Snapshot snapshot, Rollback rollback, Reset
        reset) {
        vector<ll> res(qs.size());
        sort(all(qs), [&](const Query& a, const Query&

```

```

b) {
    if (a.l / bs != b.l / bs) return a.l / bs < b
    .l / bs;
    return a.r < b.r;
});
int block = -1, cur_r = -1;
for (const Query& q : qs) {
    int b = q.l / bs;
    int block_end = min(n - 1, (b + 1) * bs - 1);
    if (b != block) {
        block = b;
        cur_r = block_end; // rango persistente
        vacio
        reset();
    }
    if (q.r <= block_end) {
        // query contenida en el bloque: brute
        force sobre estructura vacia
        snapshot();
        for (int i = q.l; i <= q.r; i++) add(i);
        res[q.id] = current();
        rollback();
    } else {
        while (cur_r < q.r) add(++cur_r);
        snapshot();
        for (int i = block_end; i >= q.l; i--) add(
i);
        res[q.id] = current();
        rollback();
    }
}
return res;
}
};

```

Strings

```

/**
 * Uso: to_lower(s), to_upper(s), trim(s),
 * replace_all(s, "a", "b"), split(s, ','), join(v
, " ");
 * Utilidades de procesamiento de strings:
 * mayusculas/minusculas, recorte, reemplazo,
 * division y union.
 * Complejidad: O(N) por operacion.
 */
void to_lower(string& s) { transform(all(s), s.
begin(), ::tolower); }
void to_upper(string& s) { transform(all(s), s.
begin(), ::toupper); }
void trim(string& s) {
    s.erase(s.begin(), find_if(all(s), [](unsigned
char c) { return !isspace(c); }));
    s.erase(find_if(rall(s), [](unsigned char c) {
return !isspace(c); }).base(), s.end());
}
void replace_all(string& s, const string& old,
const string& nw) {
    size_t pos = 0;
    while ((pos = s.find(old, pos)) != string::npos)
        s.replace(pos, old.size(), nw), pos += nw.size
();
}
vector<string> split(const string& s, char delim) {
    vector<string> res;
    string item;
    istringstream iss(s);

```

```

while (getline(iss, item, delim)) res.pb(item);
return res;
}
string join(const vector<string>& v, const string&
delim) {
    string res;
    for (int i = 0; i < v.size(); i++) {
        res += v[i];
        if (i + 1 < v.size()) res += delim;
    }
    return res;
}

/**
 * Uso: auto pi = prefix_function(s);
 * Funcion prefijo de KMP: pi[i] es el prefijo propio
 * mas largo que tambien es sufixo de s[0..i].
 * Complejidad: O(N).
 */
vi prefix_function(const string& s) {
    int n = s.size();
    vi pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

/**
 * Uso: auto z = z_function(s);
 * Funcion Z: z[i] es la longitud del prefijo de s que
 * coincide con el sufixo que inicia en i.
 * Complejidad: O(N).
 */
vi z_function(const string& s) {
    int n = s.size(), l = 0, r = 0;
    vi z(n, 0);
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}

/**
 * Uso: string res = remove_all_occurrences(s, p);
 * Elimina todas las apariciones de un patron p en
 * s, incluidas las encadenadas.
 * Complejidad: O(N + M) con N = |s| y M = |p|.
 */
string remove_all_occurrences(const string& s,
const string& p) {
    vi pi = prefix_function(p);
    string res;
    vi state(s.size() + 1, 0);
    for (char c : s) {
        res += c;
        int j = state[res.size() - 1];
        while (j > 0 && p[j] != c) j = pi[j - 1];
        if (p[j] == c) j++;
        state[res.size()] = j;
        if (j == p.size()) res.resize(res.size() - p.

```

```

    size());
}
return res;
}

/**
 * Uso[0]: auto occ = rabin_karp(text, pattern);
 * Encuentra las posiciones donde comienza un
 * patron usando hashing (rolling hash).
 * Complejidad:  $O(T+S)$  con  $T = |text|$  y  $S = |pattern|$ .
 */
vi rabin_karp(const string& text, const string&
    pattern) {
    const ll P = 31, M = 1e9 + 9;
    int S = pattern.size(), T = text.size();
    if (!S || S > T) return {};
    vll p_pow(max(S, T));
    p_pow[0] = 1;
    for (int i = 1; i < p_pow.size(); i++) p_pow[i] =
        p_pow[i - 1] * P % M;
    vll h(T + 1, 0);
    for (int i = 0; i < T; i++) h[i + 1] = (h[i] + (
        text[i] - 'a' + 1) * p_pow[i]) % M;
    ll hs = 0;
    for (int i = 0; i < S; i++) hs = (hs + (pattern[i]
        - 'a' + 1) * p_pow[i]) % M;
    vi occ;
    for (int i = 0; i + S - 1 < T; i++) {
        ll cur = (h[i + S] + M - h[i]) % M;
        if (cur == hs * p_pow[i] % M) occ.pb(i);
    }
    return occ;
}

/**
 * Uso: auto p = manacher(s);
 * Radios de los palindromos en cada posicion (
 * incluyendo centros pares); el maximo da el
 * palindromo mas largo.
 * Complejidad:  $O(N)$ .
 */
vi manacher(const string& s) {
    string t = "^#";
    for (char c : s) t += c, t += '#';
    t += '$';
    int n = t.size(), c = 0, r = 0;
    vi p(n, 0);
    for (int i = 1; i < n - 1; i++) {
        int mir = 2 * c - i;
        p[i] = r > i ? min(r - i, p[mir]) : 0;
        while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) p[i]
            ++;
        if (i + p[i] > r) c = i, r = i + p[i];
    }
    return vi(p.begin() + 1, p.end() - 1);
}

/**
 * Uso[0]: StringHash sh(s); sh.get_hash(l, r);
 * Hash doble de substrings para compararlos en
 *  $O(1)$ .
 * Complejidad: preproceso  $O(N)$ , consulta  $O(1)$ .
 */
struct StringHash {
    static constexpr ll MOD1 = 1000000007LL; // 1e9 +
    static constexpr ll MOD2 = 1000000009LL; // 1e9 +

```

```

    static constexpr ll BASE1 = 31;
    static constexpr ll BASE2 = 37;
    string s;
    int n;
    vll h1, h2, p1, p2;
    StringHash(const string& s) : s(s), n(s.size()),
        h1(n + 1, 0), h2(n + 1, 0), p1(n + 1, 1), p2(n
        + 1, 1) {
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * BASE1 + (s[i] - 'a' + 1)
            ) % MOD1;
            h2[i + 1] = (h2[i] * BASE2 + (s[i] - 'a' + 1)
            ) % MOD2;
            p1[i + 1] = p1[i] * BASE1 % MOD1;
            p2[i + 1] = p2[i] * BASE2 % MOD2;
        }
    }
    uint64_t get_hash(int l, int r) {
        ll x1 = (h1[r + 1] - h1[l] * p1[r - l + 1] %
        MOD1 + MOD1) % MOD1;
        ll x2 = (h2[r + 1] - h2[l] * p2[r - l + 1] %
        MOD2 + MOD2) % MOD2;
        return (uint64_t)x1 << 32 | (uint32_t)x2;
    }
};

/**
 * Uso: Trie t; t.insert(s); t.search(s);
 * Trie (arbol de prefijos) sobre el alfabeto de
 * minúsculas con insercion y busqueda de palabras
 * .
 * Complejidad: insercion y busqueda  $O(L)$  con  $L$  la
 * longitud de la cadena.
 */
struct Trie {
    struct Node { int next[26] = {}; bool is_end =
        false; int count = 0; };
    vector<Node> t;
    Trie() { t.emplace_back(); }
    void insert(const string& s) {
        int v = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!t[v].next[c]) t[v].next[c] = t.size(), t
                .emplace_back();
            v = t[v].next[c];
            t[v].count++;
        }
        t[v].is_end = true;
    }
    bool search(const string& s) {
        int v = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!t[v].next[c]) return false;
            v = t[v].next[c];
        }
        return t[v].is_end;
    }
};

/**
 * Uso[0]: AhoCorasick ac; ac.insert(p); ac.build()
 * ; ac.count_occurrences(text); ac.
 * first_occurrences(text);

```

```

* Automata Aho-Corasick para buscar multiples
patrones en un texto: conteo y primera
aparicion de cada uno.
* Complejidad: construccion  $O(\sum |p_i| \cdot 26)$ ,
procesamiento del texto  $O(T)$ .
*/
struct AhoCorasick {
    struct Node {
        int next[26] = {};
        int link = 0;
        vi word_ids;
    };
    vector<Node> t;
    vi bfs_order, pat_len;
    int num_patterns = 0;
    AhoCorasick() { t.emplace_back(); }
    int insert(const string& s) {
        int v = 0, id = num_patterns++;
        pat_len.pb(s.size());
        for (char ch : s) {
            int c = ch - 'a';
            if (!t[v].next[c]) t[v].next[c] = t.size(), t
                .emplace_back();
            v = t[v].next[c];
        }
        t[v].word_ids.pb(id);
        return id;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < 26; c++) {
            if (t[0].next[c]) q.push(t[0].next[c]);
            else t[0].next[c] = 0;
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            bfs_order.pb(u);
            for (int c = 0; c < 26; c++) {
                int v = t[u].next[c];
                if (v) t[v].link = t[t[u].link].next[c], q.
                    push(v);
                else t[u].next[c] = t[t[u].link].next[c];
            }
        }
    }
    // Numero de ocurrencias de cada patron en text
    vi count_occurrences(const string& text) {
        vi freq(t.size(), 0);
        int u = 0;
        for (char ch : text) u = t[u].next[ch - 'a'],
            freq[u]++;
        for (int i = bfs_order.size() - 1; i >= 0; i--)
        {
            int cur = bfs_order[i];
            freq[t[cur].link] += freq[cur];
        }
        vi ans(num_patterns, 0);
        for (int i = 1; i < t.size(); i++) if (freq[i])
            for (int id : t[i].word_ids) ans[id] += freq[i];
        return ans;
    }
    // Posicion de la primera ocurrencia de cada
    patron (o -1)
    vi first_occurrences(const string& text) {
        vi first(t.size(), INF);

```

```

        int u = 0;
        for (int i = 0; i < text.size(); i++) {
            u = t[u].next[text[i] - 'a'];
            first[u] = min(first[u], i);
        }
        for (int i = bfs_order.size() - 1; i >= 0; i--)
        {
            int cur = bfs_order[i];
            first[t[cur].link] = min(first[t[cur].link],
                first[cur]);
        }
        vi ans(num_patterns, -1);
        for (int i = 1; i < t.size(); i++) if (first[i]
            != INF)
            for (int id : t[i].word_ids) {
                int st = first[i] - pat_len[id] + 1;
                ans[id] = ans[id] == -1 ? st : min(ans[id],
                    st);
            }
        return ans;
    }
};

/**
* Uso[0]: SuffixAutomaton sam(s); sam.
distinct_substrings(); sam.count_occurrences(p)
; sam.first_occurrence(p);
* Automata de sufixos: substrings distintos,
ocurrencias y primera aparicion de un patron.
* Complejidad: construccion  $O(N \cdot 26)$ , consultas
 $O(|p|)$ .
*/
struct SuffixAutomaton {
    struct Node {
        int len = 0, link = -1, next[26] = {};
        int firstpos = -1;
        int cnt = 0;
    };
    vector<Node> t;
    int last;
    bool occ_computed = false;
    SuffixAutomaton(const string& s = "") {
        t.reserve(s.size() * 2 + 1);
        t.emplace_back();
        last = 0;
        for (char c : s) extend(c - 'a');
    }
    void extend(int c) {
        int cur = t.size();
        t.emplace_back();
        t[cur].len = t[last].len + 1;
        t[cur].firstpos = t[cur].len - 1;
        t[cur].cnt = 1;
        int p = last;
        while (p != -1 && !t[p].next[c]) t[p].next[c] =
            cur, p = t[p].link;
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].next[c];
            if (t[p].len + 1 == t[q].len) t[cur].link = q;
        }
        else {
            int clone = t.size();
            t.push_back(t[q]);
            t[clone].len = t[p].len + 1;
            t[clone].cnt = 0;

```

```

        while (p != -1 && t[p].next[c] == q) t[p].
next[c] = clone, p = t[p].link;
        t[q].link = t[cur].link = clone;
    }
    last = cur;
}
void compute_occurrences() {
    if (occ_computed) return;
    occ_computed = true;
    int n = t.size();
    vi c(n + 1, 0), order(n);
    for (int i = 0; i < n; i++) c[t[i].len]++;
    for (int i = 1; i <= n; i++) c[i] += c[i - 1];
    for (int i = 0; i < n; i++) order[--c[t[i].len]] = i;
    for (int i = n - 1; i > 0; i--) {
        int u = order[i];
        if (t[u].link != -1) t[t[u].link].cnt += t[u].cnt;
    }
}
ll distinct_substrings() {
    ll ans = 0;
    for (int i = 1; i < t.size(); i++) ans += t[i].len - t[t[i].link].len;
    return ans;
}
int first_occurrence(const string& p) {
    int u = 0;
    for (char ch : p) {
        int c = ch - 'a';
        if (!t[u].next[c]) return -1;
        u = t[u].next[c];
    }
    return t[u].firstpos - p.size() + 1;
}
ll count_occurrences(const string& p) {
    compute_occurrences();
    int u = 0;
    for (char ch : p) {
        int c = ch - 'a';
        if (!t[u].next[c]) return 0;
        u = t[u].next[c];
    }
    return t[u].cnt;
}
};

/**
 * Uso[0]: SuffixArray sa(s); sa.p (sufijos ordenados); sa.get_lcp(i, j);
 * Arreglo de sufijos con tabla RMQ de lcp para consultar el prefijo comun mas largo de dos sufijos en O(1).
 * Complejidad: construccion O(N log N), consulta lcp O(1).
 */
struct SuffixArray {
    string s;
    int n, log_n;
    vi p, c, lcp, rank;
    vector<vi> st;
    SuffixArray(string s) : s(s + "$"), n(s.size()) {
        build_sa(); build_lcp(); build_rmq();
    }
    void build_sa() {

```

```

        const int alphabet = 256;
        p.assign(n, 0); c.assign(n, 0);
        vi cnt(max(alphabet, n), 0), p_new(n), c_new(n);
        for (int i = 0; i < n; i++) cnt[s[i]]++;
        for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
        for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
        c[p[0]] = 0;
        int classes = 1;
        for (int i = 1; i < n; i++) {
            if (s[p[i]] != s[p[i - 1]]) classes++;
            c[p[i]] = classes - 1;
        }
        for (int k = 0; (1 << k) < n; k++) {
            for (int i = 0; i < n; i++) {
                p_new[i] = p[i] - (1 << k);
                if (p_new[i] < 0) p_new[i] += n;
            }
            fill(cnt.begin(), cnt.begin() + classes, 0);
            for (int i = 0; i < n; i++) cnt[c[p_new[i]]]++;
            for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) p[--cnt[c[p_new[i]]]] = p_new[i];
            c_new[p[0]] = 0;
            classes = 1;
            for (int i = 1; i < n; i++) {
                pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << k)) % n]};
                pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << k)) % n]};
                if (cur != prev) classes++;
                c_new[p[i]] = classes - 1;
            }
            c.swap(c_new);
        }
    }
}
void build_lcp() {
    lcp.assign(n, 0);
    rank.assign(n, 0);
    for (int i = 0; i < n; i++) rank[p[i]] = i;
    int k = 0;
    for (int i = 0; i < n - 1; i++) {
        int pi = rank[i], j = p[pi - 1];
        while (s[i + k] == s[j + k]) k++;
        lcp[pi] = k;
        if (k) k--;
    }
}
void build_rmq() {
    log_n = log2(n) + 1;
    st.assign(n, vi(log_n));
    for (int i = 0; i < n; i++) st[i][0] = lcp[i];
    for (int j = 1; j < log_n; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            st[i][j] = min(st[i][j - 1], st[i + (1 << j) - 1][j - 1]);
}
// LCP de los sufijos que empiezan en i y j
int get_lcp(int i, int j) {
    if (i == j) return n - 1 - i;
    int u = rank[i], v = rank[j];
    if (u > v) swap(u, v);

```

```

    u++;
    int k = log2(v - u + 1);
    return min(st[u][k], st[v - (1 << k) + 1][k]);
}
};

```

```

/**
 * Uso[0]: PalindromicTree pt(s); pt.
 * count_palindromes(); pt.compute_occurrences();
 * Arbol de palindromos (eertree): todos los
 * palindromos distintos, sus ocurrencias y su
 * primera posicion.
 * Complejidad: construccion  $O(N \cdot 26)$ , ocurrencias
 *  $O(N)$ .
 */
struct PalindromicTree {
    struct Node {
        int next[26] = {};
        int len = 0, link = 0;
        ll occ = 0;
        int first_pos = -1; // posicion donde aparece
        por primera vez
    };
    vector<Node> t;
    string s; // s[0] = sentinel, nunca coincide con
    una letra
    int last;
    PalindromicTree(const string& str = "") {
        t.emplace_back(); t.emplace_back();
        t[0].len = -1; t[0].link = 0; // raiz
        imaginaria (len -1)
        t[1].len = 0; t[1].link = 0; // raiz del vacio
        (len 0)
        last = 1;
        s = "$";
        for (char c : str) add(c);
    }
    int get_link(int v, int pos) {
        while (true) {
            int idx = pos - t[v].len - 1;
            if (idx >= 0 && s[idx] == s[pos]) break;
            v = t[v].link;
        }
        return v;
    }
    void add(char c) {
        s += c;
        int pos = s.size() - 1;
        int cur = get_link(last, pos);
        int idx = c - 'a';
        if (!t[cur].next[idx]) {
            int now = t.size();
            t.emplace_back();
            t[now].len = t[cur].len + 2;
            t[now].first_pos = pos - t[now].len; // 0-
            indexado
            t[now].link = t[now].len == 1 ? 1 : t[
            get_link(t[cur].link, pos)].next[idx];
            t[cur].next[idx] = now;
        }
        last = t[cur].next[idx];
        t[last].occ++;
    }
    ll count_palindromes() { return t.size() - 2; }
    // palindromos distintos
    void compute_occurrences() { // occ[u] = veces
    que aparece el palindromo u

```

```

        for (int i = t.size() - 1; i > 1; i--) t[t[i].
        link].occ += t[i].occ;
    }
};

```

Dynamic Programming

```

/**
 * Uso: LiChaoTree cht; cht.add(m, c); ll v = cht.
 * query(x);
 * Arbol de Li Chao para consultar el minimo de
 *  $m \cdot x + c$  entre las rectas insertadas (cambiar <
 * por > para max).
 * Complejidad: insercion y consulta  $O(\log R)$  con  $R$ 
 * el tamano del rango de  $x$ .
 */
struct LiChaoTree {
    struct Line {
        ll m, c;
        Line(ll m = 0, ll c = LINF) : m(m), c(c) {}
        ll eval(ll x) const { return m * x + c; }
    };
    struct Node { Line line; int lc = -1, rc = -1; };
    vector<Node> st;
    ll min_x, max_x;
    LiChaoTree(ll mn = -1e9, ll mx = 1e9) : min_x(mn)
    , max_x(mx) { st.emplace_back(); }
    void add_line(Line nw, int v, ll l, ll r) {
        ll mid = l + (r - l) / 2;
        bool left = nw.eval(l) < st[v].line.eval(l);
        bool midb = nw.eval(mid) < st[v].line.eval(mid);
        if (midb) swap(st[v].line, nw);
        if (l == r) return;
        if (left != midb) {
            if (st[v].lc == -1) st[v].lc = st.size(), st.
            emplace_back();
            add_line(nw, st[v].lc, l, mid);
        } else {
            if (st[v].rc == -1) st[v].rc = st.size(), st.
            emplace_back();
            add_line(nw, st[v].rc, mid + 1, r);
        }
    }
    void add(ll m, ll c) { add_line(Line(m, c), 0,
    min_x, max_x); }
    ll query(ll x, int v, ll l, ll r) {
        if (v == -1) return LINF;
        ll res = st[v].line.eval(x);
        if (l == r) return res;
        ll mid = l + (r - l) / 2;
        if (x <= mid) return min(res, query(x, st[v].lc
        , l, mid));
        return min(res, query(x, st[v].rc, mid + 1, r))
        ;
    }
    ll query(ll x) { return query(x, 0, min_x, max_x)
    ; }
};

/**
 * Uso: dnc_dp(1, n, 0, n, dp_prev, dp_curr, cost);
 * Divide & conquer DP: resuelve
 *  $dp[i][j] = \min_{k < j} (dp[i-1][k] + cost(k+1, j))$  cuando la
 * decision optima es monotona.
 * Ejemplo de cost [costo de un segmento, en  $O(1)$ ]:
 * auto cost = [&](int l, int r){ return pref[r]

```

```

- pref[l - 1]; };
* Indexación: 0-indexado; para particionar en k
  grupos, dp_prev[0] = 0.
* Complejidad:  $O(KN \log N)$  con cost evaluada en  $O(1)$ .
*/
template<class CostFunc>
void dnc_dp(int l, int r, int optl, int optr, const
  vll& prev, vll& curr, CostFunc& cost) {
  if (l > r) return;
  int mid = (l + r) / 2;
  pair<ll, int> best = {LINF, -1};
  for (int k = optl; k <= min(mid - 1, optr); k++)
  {
    ll cur = prev[k] == LINF ? LINF : prev[k] +
      cost(k + 1, mid);
    if (cur < best.fi) best = {cur, k};
  }
  curr[mid] = best.fi;
  int opt = best.se == -1 ? optl : best.se;
  dnc_dp(l, mid - 1, optl, opt, prev, curr, cost);
  dnc_dp(mid + 1, r, opt, optr, prev, curr, cost);
}

/**
* Uso: auto dp = knuth_dp(n, cost);
* Optimización de Knuth para DP de intervalos
  dp[i][j] = min_k (dp[i][k] + dp[k][j]) + cost(i, j) con decision
  monotona.
* Complejidad:  $O(N^2)$ .
*/
template<class CostFunc>
vector<vll> knuth_dp(int n, CostFunc& cost) {
  vector<vll> dp(n + 1, vll(n + 1, 0));
  vector<vi> opt(n + 1, vi(n + 1, 0));
  for (int i = 0; i < n; i++) opt[i][i + 1] = i;
  for (int len = 2; len <= n; len++)
    for (int i = 0; i + len <= n; i++) {
      int j = i + len;
      dp[i][j] = LINF;
      for (int k = opt[i][j - 1]; k <= min(j - 1,
        opt[i + 1][j]); k++) {
        ll cur = dp[i][k] + dp[k][j] + cost(i, j);
        if (cur < dp[i][j]) dp[i][j] = cur, opt[i][
          j] = k;
      }
    }
  return dp;
}

/**
* Uso: DigitDP::solve(R) - DigitDP::solve(L-1);
* DP sobre digitos para contar numeros en  $[0, N]$  que
  cumplen condiciones; ajustar memo y
  transiciones segun el problema.
* Complejidad:  $O(\text{digitos} \times \text{estados})$  por llamada.
*/
namespace DigitDP {
  string s;
  ll memo[20][2][2]; // agregar estados segun
    problema
  ll dp(int idx, bool tight, bool lz) {
    if (idx == s.size()) return 1; // 1 si el
      numero es valido
    if (memo[idx][tight][lz] != -1) return memo[idx]
      [tight][lz];
    int lim = tight ? s[idx] - '0' : 9;
    ll ans = 0;

```

```

    for (int d = 0; d <= lim; d++)
      ans += dp(idx + 1, tight && d == lim, lz && d
        == 0);
    return memo[idx][tight][lz] = ans;
  }
  ll solve(ll n) {
    if (n < 0) return 0;
    s = to_string(n);
    memset(memo, -1, sizeof(memo));
    return dp(0, true, true);
  }
}

/**
* Uso: sos_dp(dp, n_bits);
* Suma sobre subconjuntos: dp[mask] acumula la suma
  de todos sus subconjuntos (sum over subsets).
* Complejidad:  $O(N \cdot 2^N)$ .
*/
void sos_dp(vll& dp, int n) {
  for (int i = 0; i < n; i++)
    for (int mask = 0; mask < (1 << n); mask++)
      if (mask & (1 << i)) dp[mask] += dp[mask ^ (1
        << i)];
}

/**
* Uso: auto [len, seq] = lis(a);
* Subsecuencia creciente mas larga estricta, con
  reconstrucción de la secuencia.
* Complejidad:  $O(N \log N)$ .
*/
pair<int, vi> lis(const vi& a) {
  int n = a.size();
  vi d, pos, parent(n, -1);
  for (int i = 0; i < n; i++) {
    auto it = lower_bound(all(d), a[i]);
    int idx = it - d.begin();
    if (it == d.end()) d.pb(a[i]), pos.pb(i);
    else *it = a[i], pos[idx] = i;
    if (idx > 0) parent[i] = pos[idx - 1];
  }
  vi seq;
  for (int cur = pos.empty() ? -1 : pos.back(); cur
    != -1; cur = parent[cur]) seq.pb(a[cur]);
  reverse(all(seq));
  return {d.size(), seq};
}

/**
* Uso: bitset_knapsack(pesos, target);
* Determina si un subconjunto de pesos suma
  exactamente un valor objetivo mediante bitsets.
* Complejidad:  $O(N \cdot S/64)$  con S la suma maxima ( $10^5$ 
  fija en MAX_SUM).
*/
const int MAX_SUM = 1e5 + 5;
bool bitset_knapsack(const vi& weights, int target)
{
  bitset<MAX_SUM> dp;
  dp[0] = 1;
  for (int w : weights) dp |= dp << w;
  return dp[target];
}

/**

```

```

* Uso: ll ans = WQS::solve(check, K);
* Búsqueda binaria de Lagrange (Alien's trick)
  para optimizar un DP con la restricción de usar
  exactamente K elementos o particiones.
* Complejidad:  $O(\log R \cdot f)$  con  $f$  el costo del DP por
  evaluación y  $R$  el rango de la penalidad.
*/
namespace WQS {
    template <class F>
    ll solve(F check, int K) {
        ll lo = -LINF, hi = LINF;
        while (lo + 1 < hi) {
            ll mid = lo + (hi - lo) / 2;
            if (check(mid).se >= K) lo = mid;
            else hi = mid;
        }
        auto [c, k] = check(lo);
        return c - lo * (ll)K; // quita la penalidad de
        los K elementos
    }
}

/**
* Uso: CHTDeque cht; cht.add(m, c); cht.query(x);
  // y cht.query_bs(x) para x arbitraria
* Convex hull trick con deque para mínimos  $m \cdot x + c$ 
  con pendientes monotonas y consultas con  $x$ 
  creciente.
* Complejidad: inserción y consulta amortizadas
   $O(1)$ ; consulta binaria  $O(\log N)$ .
*/
struct CHTDeque {
    struct Line { ll m, c; };
    deque<Line> dq;
    int slope_inc = 0; // +1 pendientes crecientes,
    -1 decrecientes (auto-detectado)
    static ll eval(const Line& l, ll x) { return l.m
        * x + l.c; }
    // true si l2 queda obsoleta entre l1 y l3
    bool is_bad(const Line& l1, const Line& l2, const
    Line& l3) {
        if (slope_inc > 0) return (__int128)(l1.c - l2.
        c) * (l3.m - l2.m) <= (__int128)(l2.c - l3.c) *
        (l2.m - l1.m);
        return (__int128)(l3.c - l1.c) * (l1.m - l2.m)
        <= (__int128)(l2.c - l1.c) * (l1.m - l3.m);
    }
    void add(ll m, ll c) {
        if (!dq.empty() && dq.back().m == m) {
            if (dq.back().c <= c) return; // la nueva no
            aporta (min)
            dq.pop_back();
        }
        if (dq.size() >= 2 && slope_inc == 0) slope_inc
        = dq.back().m < m ? 1 : -1;
        Line nw{m, c};
        while (dq.size() >= 2 && is_bad(dq[dq.size() -
        2], dq.back(), nw)) dq.pop_back();
        dq.push_back(nw);
    }
    ll query(ll x) { // x debe ser no decreciente
        if (slope_inc > 0) { // pendientes crecientes:
            optimo va del fondo al frente
            while (dq.size() >= 2 && eval(dq.back(), x)
            >= eval(dq[dq.size() - 2], x)) dq.pop_back();
            return eval(dq.back(), x);

```

```

        }
        while (dq.size() >= 2 && eval(dq.front(), x) >=
        eval(dq[1], x)) dq.pop_front();
        return eval(dq.front(), x);
    }
    ll query_bs(ll x) { // x arbitraria
        int lo = 0, hi = (int)dq.size() - 1;
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (eval(dq[mid], x) <= eval(dq[mid + 1], x))
                hi = mid;
            else lo = mid + 1;
        }
        return eval(dq[lo], x);
    }
};

```

Mathematics

```

namespace MathAlgo {
    /**
    * Uso: modpow(a, b, mod), modinv(a, mod),
    ext_gcd(a, b, x, y);
    * Potencia modular, inverso modular y euclides
    extendido ( $ax + by = \gcd(a, b)$ ).
    * Complejidad:  $O(\log b)$  por operación.
    */
    ll modpow(ll a, ll b, ll m = MOD) {
        ll r = 1; a %= m;
        while (b) { if (b & 1) r = r * a % m; a = a * a
        % m; b >>= 1; }
        return r;
    }
    // Euclides extendido: resuelve  $ax + by = \gcd(a, b)$ 
    ll ext_gcd(ll a, ll b, ll& x, ll& y) {
        if (!b) { x = 1; y = 0; return a; }
        ll x1, y1, d = ext_gcd(b, a % b, x1, y1);
        x = y1; y = x1 - y1 * (a / b); return d;
    }
    /**
    * Uso: auto [x, lcm] = modinv(a, mod);
    * Inverso modular de a modulo m (exige
     $\gcd(a, m) = 1$ ).
    * Complejidad:  $O(\log m)$ .
    */
    ll modinv(ll a, ll m = MOD) {
        ll x, y; assert(ext_gcd(a, m, x, y) == 1);
        return (x % m + m) % m;
    }
    /**
    * Uso: auto [x, lcm] = crt(a1, m1, a2, m2);
    * Teorema chino del resto para dos congruencias;
    devuelve  $\{-1, -1\}$  si no tienen solución.
    * Complejidad:  $O(\log \min(m_1, m_2))$ .
    */
    pair<ll, ll> crt(ll a1, ll m1, ll a2, ll m2) {
        ll p, q, g = ext_gcd(m1, m2, p, q);
        if (a1 % g != a2 % g) return {-1, -1};
        ll lcm = (m1 / g) * m2;
        ll res = a1 + (ll)((__int128_t)(a2 - a1) / g *
        p % (m2 / g)) * m1;
        return {(res % lcm + lcm) % lcm, lcm};
    }
    /**
    * Uso: auto [x, lcm] = solve_crt_array(A, M);

```

```

* Teorema chino del resto para un arreglo de congruencias; devuelve {-1,-1} si alguna es inconsistente.
* Complejidad:  $O(K \log m)$  con  $K$  el numero de congruencias.
*/
pair<ll, ll> solve_crt_array(const vector<ll>& A,
    const vector<ll>& M) {
    ll ans = A[0];
    ll lcm = M[0];
    for (int i = 1; i < A.size(); i++) {
        auto res = crt(ans, lcm, A[i], M[i]);
        if (res.first == -1) return {-1, -1}; //
        No hay solución
        ans = res.first;
        lcm = res.second;
    }
    return {ans, lcm};
}

/**
* Uso: Sieve sv(n); sv.primes; sv.spf; sv.mu; sv
.phi; sv.factorize(x);
* Criba lineal con lista de primos, menor factor primo, funcion de Mobius y totient de Euler.
* Complejidad: criba  $O(N)$ , factorizacion de un valor  $O(\log x)$ .
*/
struct Sieve {
    vector<int> primes, spf, mu, phi;
    Sieve(int n) {
        spf.assign(n + 1, 0); mu.assign(n + 1, 0);
        phi.assign(n + 1, 0);
        mu[1] = phi[1] = 1;
        for (int i = 2; i <= n; i++) {
            if (!spf[i]) spf[i] = i, primes.pb(i), mu[i] = -1, phi[i] = i - 1;
            for (int p : primes) {
                if (p > spf[i] || 1LL * i * p > n) break;
                spf[i * p] = p;
                if (i % p == 0) { mu[i * p] = 0; phi[i * p] = phi[i] * p; break; }
                mu[i * p] = -mu[i]; phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
    vector<int> factorize(int x) { // factores primos con multiplicidad
        vector<int> r;
        while (x > 1) r.pb(spf[x]), x /= spf[x];
        return r;
    }
};

/**
* Uso: Combinatorics c(n, mod); c.nCr(n, r); c.catalan(n);
* Factoriales e inversos para combinar y numeros de Catalan modulo primo.
* Complejidad: preproceso  $O(N)$ , consulta  $O(1)$ .
*/
struct Combinatorics {
    vector<ll> fact, invfact;
    Combinatorics(int n, ll mod = MOD) {
        fact.assign(n + 1, 1); invfact.assign(n + 1, 1);
        for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % mod;
        invfact[n] = modinv(fact[n], mod);
        for (int i = n - 1; i >= 0; i--) invfact[i] = invfact[i + 1] * (i + 1) % mod;
        this->mod = mod;
    }
    ll mod;
    ll nCr(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] * invfact[r] % mod * invfact[n - r] % mod;
    }
    ll catalan(int n) { return nCr(2 * n, n) * modinv(n + 1, mod) % mod; }
};

/**
* Uso: gauss(matriz_extendida, soluciones);
* Eliminacion gaussiana con pivoteo para sistemas lineales; devuelve 1, INF o 0 segun el numero de soluciones.
* Complejidad:  $O(n \cdot m^2)$ .
*/
int gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int sel = row;
        for (int i = row; i < n; i++) if (fabs(a[i][col]) > fabs(a[sel][col])) sel = i;
        if (fabs(a[sel][col]) < EPS) continue;
        for (int i = col; i <= m; i++) swap(a[sel][i], a[row][i]);
        where[col] = row;
        for (int i = 0; i < n; i++) if (i != row) {
            double c = a[i][col] / a[row][col];
            for (int j = col; j <= m; j++) a[i][j] -= a[row][j] * c;
        }
        row++;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++) if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i = 0; i < n; i++) {
        double s = 0;
        for (int j = 0; j < m; j++) s += ans[j] * a[i][j];
        if (fabs(s - a[i][m]) > EPS) return 0;
    }
    for (int i = 0; i < m; i++) if (where[i] == -1) return INF;
    return 1;
}

/**
* Uso: auto c = multiply(a, b); // producto de polinomios (recomendado)
* Transformada rapida de Fourier iterativa y multiplicacion exacta de polinomios con coeficientes enteros.
* fft(vec, invert) modifica el vector en su lugar : invert = false aplica la transformada directa e invert = true la inversa.

```

```

* En general usa multiply(a, b), que alinea
ambos polinomios a potencia de dos, transforma,
multiplica punto a punto e invierte; no hace
falta inicializar el tamaño.
* Complejidad:  $O(N \log N)$  con  $N$  potencia de dos.
*/
using cd = complex<double>;
// FFT iterativa, invert = true para transformada
// inversa
void fft(vector<cd>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * acos(-1.0) / len * (invert ?
        -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j], v = a[i + j + len / 2] *
                w;
                a[i + j] = u + v; a[i + j + len / 2] = u
                - v; w *= wlen;
            }
        }
    }
    if (invert) for (cd& x : a) x /= n;
}

// Multiplicacion de polinomios (coeficientes
// enteros)  $O(n \log n)$ 
vector<int> multiply(vector<int> const& a, vector
<int> const& b) {
    vector<cd> fa(all(a)), fb(all(b));
    int n = 1;
    while (n < a.size() + b.size()) n <= 1;
    fa.resize(n); fb.resize(n);
    fft(fa, false); fft(fb, false);
    for (int i = 0; i < n; i++) fa[i] *= fb[i];
    fft(fa, true);
    vector<int> res(n);
    for (int i = 0; i < n; i++) res[i] = round(fa[i]
    ].real());
    while (res.size() > 1 && !res.back()) res.
    pop_back();
    return res;
}

/**
 * Uso: auto c = multiply_mod(a, b); // producto
 * de polinomios mod MOD (recomendado)
 * Transformada numerica rapida (NTT) y
 * multiplicacion de polinomios con coeficientes
 * modulares.
 * ntt(vec, invert) modifica el vector en su lugar
 * : invert = false directa, invert = true inversa
 * . Requiere que el tamaño sea potencia de dos.
 * En general usa multiply_mod(a, b), que alinea
 * a potencia de dos y hace todo el proceso; no
 * hace falta inicializar el tamaño.
 * Complejidad:  $O(N \log N)$  con  $N$  potencia de dos.
 */
void ntt(vector<ll>& a, bool invert) {

```

```

    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        ll wlen = modpow(3, (MOD - 1) / len);
        if (invert) wlen = modinv(wlen);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = a[i + j + len / 2] *
                w % MOD;
                a[i + j] = (u + v) % MOD;
                a[i + j + len / 2] = (u - v + MOD) % MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (invert) {
        ll ninv = modinv(n);
        for (ll& x : a) x = x * ninv % MOD;
    }
}

vector<ll> multiply_mod(vector<ll> const& a,
vector<ll> const& b) {
    vector<ll> fa(all(a)), fb(all(b));
    int n = 1;
    while (n < a.size() + b.size()) n <= 1;
    fa.resize(n); fb.resize(n);
    ntt(fa, false); ntt(fb, false);
    for (int i = 0; i < n; i++) fa[i] = fa[i] * fb[
    i] % MOD;
    ntt(fa, true);
    while (fa.size() > 1 && !fa.back()) fa.pop_back
    ();
    return fa;
}

/**
 * Uso: nim_game(pilas);
 * Juego de Nim: devuelve true si el primer
 * jugador gana (el xor de las pilas es distinto
 * de 0).
 * Complejidad:  $O(N)$ .
 */
bool nim_game(const vi& piles) {
    int x = 0;
    for (int p : piles) x ^= p;
    return x != 0;
}

/**
 * Uso: sum_n(n), sum_squares(n), sum_cubes(n),
 * geom_sum(a, r, n);
 * Sumatorias cerradas modulo M: aritmetica
 *  $\sum_{i=1}^n i$ , de cuadrados  $\sum_{i=1}^n i^2$ , de cubos  $\sum_{i=1}^n i^3$  y
 * geometrica  $\sum_{i=0}^{n-1} ar^i$ .
 * Complejidad:  $O(\log M)$  por llamada (inverso
 * modular).
 */
ll sum_n(ll n, ll m = MOD) { n %= m; return n * (
    n + 1) % m * modinv(2, m) % m; }
ll sum_squares(ll n, ll m = MOD) {

```

```

    n %= m;
    return n * (n + 1) % m * (2 * n + 1) % m *
    modinv(6, m) % m;
}
11 sum_cubes(11 n, 11 m = MOD) { // (n*(n+1)/2)^2
    n %= m;
    11 s = n * (n + 1) % m * modinv(2, m) % m;
    return s * s % m;
}
11 geom_sum(11 a, 11 r, 11 n, 11 m = MOD) {
    if (r == 1) return (n % m) * (a % m) % m;
    11 num = (modpow(r, n, m) - 1 + m) % m;
    return (a % m) * num % m * modinv((r - 1 + m) %
    m, m) % m;
}

/**
 * Uso: PI, to_rad(grados), to_deg(radianes);
 * Constante  $\pi$  y conversion grados/radianes para
 * trabajar con funciones trigonometricas y
 * angulos en radianes.
 * Complejidad:  $O(1)$ .
 */
const long double PI = acos(-1.0L);
long double to_rad(long double deg) { return deg
    * PI / 180.0L; }
long double to_deg(long double rad) { return rad
    * 180.0L / PI; }

/**
 * Uso: Matrix m(r, c); m.mat; m * o; m.power(p);
 * Matrix::identity(n);
 * Matriz con multiplicacion modular y
 * exponenciacion para recurrencias lineales y
 * conteo de caminos.
 * Complejidad: producto  $O(r \cdot c^2)$ , potencia
 *  $O(r^3 \log p)$ .
 */
struct Matrix {
    int r, c;
    vector<vector<ll>> mat;
    Matrix(int r, int c) : r(r), c(c), mat(r,
    vector<ll>(c, 0)) {}
    static Matrix identity(int n) {
        Matrix I(n, n);
        for (int i = 0; i < n; i++) I.mat[i][i] = 1;
        return I;
    }
    Matrix operator*(const Matrix& o) const {
        Matrix res(r, o.c);
        for (int i = 0; i < r; i++)
            for (int k = 0; k < c; k++) if (mat[i][k])
                for (int j = 0; j < o.c; j++)
                    res.mat[i][j] = (res.mat[i][j] + mat[i]
                    ][k] * o.mat[k][j]) % MOD;
        return res;
    }
    Matrix power(11 p) const {
        Matrix res = identity(r), base = *this;
        while (p) { if (p & 1) res = res * base; base =
        base * base; p >>= 1; }
        return res;
    }
};

/**

```

```

 * Uso: auto C = berlekamp_massey(sequencia);
 * Encuentra la recurrencia lineal minima de una
 * secuencia;  $C[0] \cdot S[n] + C[1] \cdot S[n-1] + \dots = 0$ .
 * Complejidad:  $O(N^2)$ .
 */
vector<ll> berlekamp_massey(const vector<ll>& s)
{
    vector<ll> C{1}, B{1};
    int L = 0; 11 m = 1, b = 1;
    for (int i = 0; i < s.size(); i++) {
        11 d = 0;
        for (int j = 0; j <= L; j++) d = (d + C[j] *
        s[i - j]) % MOD;
        if (d) {
            vector<ll> T = C;
            11 coef = MOD - d * modpow(b, MOD - 2) %
            MOD;
            while (C.size() <= B.size() + m) C.pb(0);
            for (int j = 0; j < B.size(); j++) C[j + m]
            = (C[j + m] + coef * B[j]) % MOD;
            if (2 * L <= i) L = i + 1 - L, B = T, b = d
            , m = 1;
            else m++;
        } else m++;
    }
    C.resize(L + 1);
    return C;
}

/**
 * Uso: PollardRho::is_prime(n); PollardRho::
 * factorize(n);
 * Test de primalidad Miller-Rabin y
 * factorizacion de enteros de hasta  $2^{64}$  con
 * Pollard-Rho.
 * Complejidad:  $O(n^{1/4})$  esperado por factor
 * encontrado.
 */
struct PollardRho {
    using u64 = uint64_t; using u128 = __uint128_t;
    static u64 modpow(u64 b, u64 e, u64 m) {
        u64 r = 1; b %= m;
        while (e) { if (e & 1) r = (u128)r * b % m; b =
        (u128)b * b % m; e >>= 1; }
        return r;
    }
    static bool is_prime(u64 n) {
        if (n < 2) return false;
        for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23,
        29, 31, 37}) {
            if (n == p) return true;
            if (n % p == 0) return false;
        }
        u64 d = n - 1; int s = 0;
        while (!(d & 1)) d >>= 1, s++;
        for (u64 a : {2, 3, 5, 7, 11, 13, 17, 19, 23,
        29, 31, 37}) {
            if (a >= n) continue;
            u64 x = modpow(a, d, n);
            if (x == 1 || x == n - 1) continue;
            bool comp = true;
            for (int r = 1; r < s; r++) {
                x = (u128)x * x % n;
                if (x == n - 1) { comp = false; break; }
            }
            if (comp) return false;
        }
    }
}

```

```

    return true;
}
static u64 get_factor(u64 n) {
    if (!(n & 1)) return 2;
    if (is_prime(n)) return n;
    u64 x = 2, y = 2, d = 1, c = 1;
    auto f = [&](u64 x) { return (u64)((u128)x *
x % n + c) % n); };
    while (d == 1) {
        x = f(x); y = f(f(y));
        d = gcd(x > y ? x - y : y - x, n);
        if (d == n) { x = rand() % (n - 2) + 2; y =
x; c = rand() % (n - 1) + 1; d = 1; }
    }
    return d;
}
static void factor(u64 n, map<u64, int>& f) {
    if (n == 1) return;
    if (is_prime(n)) { f[n]++; return; }
    u64 d = get_factor(n);
    factor(d, f); factor(n / d, f);
}
static map<u64, int> factorize(u64 n) { // {
    primo, exponente}
    map<u64, int> f;
    factor(n, f);
    return f;
}
};

/**
 * Uso: Lucas l(p); l.solve(n, r);
 * Combinaciones modulo un primo pequeno con n y
r potencialmente enormes (teorema de Lucas).
 * Complejidad: O(p) de preproceso, O(logp n) por
consulta.
 */
struct Lucas {
    Combinatorics comb;
    ll P;
    Lucas(ll p) : P(p), comb(p - 1, p) {}
    ll solve(ll n, ll r) {
        if (r < 0 || r > n) return 0;
        if (!r) return 1;
        if (n % P < r % P) return 0;
        return solve(n / P, r / P) * comb.nCr(n % P,
r % P) % P;
    }
};

/**
 * Uso: auto c = xor_convolution(a, b); // igual
para or_ y and_ (recomendado)
 * Transformada de Walsh-Hadamard y convoluciones
sobre operaciones de bits (xor, and, or)
modulo M.
 * fwht_xor(v, invert) modifica el vector en su
lugar: invert = false directa, invert = true
inversa; el tamano debe ser potencia de dos.
 * En general usa xor_convolution /
or_convolution / and_convolution, que alinean a
potencia de dos y hacen todo el proceso.
 * Complejidad: O(N log N).
 */
void fwht_xor(vector<ll>& a, bool invert) {
    int n = a.size();

```

```

    for (int len = 2; len <= n; len <= 1) {
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = a[i + j + len / 2];
                a[i + j] = (u + v) % MOD;
                a[i + j + len / 2] = (u - v + MOD) % MOD;
            }
        }
    }
    if (invert) {
        ll inv = modinv(n);
        for (ll& x : a) x = x * inv % MOD;
    }
}
void fwht_or(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int len = 2; len <= n; len <= 1) {
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = a[i + j + len / 2];
                if (invert) a[i + j + len / 2] = (v - u +
MOD) % MOD;
                else a[i + j + len / 2] = (u + v) % MOD;
            }
        }
    }
}
void fwht_and(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int len = 2; len <= n; len <= 1) {
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = a[i + j + len / 2];
                if (invert) a[i + j] = (u - v + MOD) %
MOD;
                else a[i + j] = (u + v) % MOD;
            }
        }
    }
}
vector<ll> xor_convolution(vector<ll> a, vector<
ll> b) {
    int n = 1;
    while (n < max(a.size(), b.size())) n <= 1;
    a.resize(n); b.resize(n);
    fwht_xor(a, false); fwht_xor(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i]
% MOD;
    fwht_xor(a, true);
    return a;
}
vector<ll> or_convolution(vector<ll> a, vector<ll>
> b) {
    int n = 1;
    while (n < max(a.size(), b.size())) n <= 1;
    a.resize(n); b.resize(n);
    fwht_or(a, false); fwht_or(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i]
% MOD;
    fwht_or(a, true);
    return a;
}
vector<ll> and_convolution(vector<ll> a, vector<
ll> b) {
    int n = 1;

```

```

while (n < max(a.size(), b.size())) n <= 1;
a.resize(n); b.resize(n);
fwht_and(a, false); fwht_and(b, false);
for (int i = 0; i < n; i++) a[i] = a[i] * b[i]
% MOD;
fwht_and(a, true);
return a;
}

/**
 * Uso: floor_sum(n, m, a, b);
 * Calcula  $\sum_{i=0}^{n-1} [(a \cdot i + b) / m]$ .
 * Complejidad:  $O(\log m)$ .
 */
11 floor_sum(11 n, 11 m, 11 a, 11 b) {
    11 ans = 0;
    while (true) {
        if (a >= m) { ans += (n - 1) * n * (a / m) /
2; a %= m; }
        if (b >= m) { ans += n * (b / m); b %= m; }
        11 y_max = a * n + b;
        if (y_max < m) break;
        n = y_max / m;
        b = y_max % m;
        swap(m, a);
    }
    return ans;
}
}

```

Geometry

```

/**
 * Uso: cmp(a, b);
 * Comparacion de numeros de punto flotante con
    tolerancia EPS; devuelve -1, 0 o 1.
 * Complejidad:  $O(1)$ .
 */
int cmp(1d x, 1d y = 0) { return x <= y + EPS ? (x
+ EPS < y ? -1 : 0) : 1; }

/**
 * Uso: Point<11> p(x, y); p+q, p-q, p*c, p/c, p==q
    ; orden lexicografico con <
 * Punto 2D generico con operadores aritmeticos,
    igualdad y orden lexicografico.
 * Complejidad:  $O(1)$  por operacion.
 */
template<typename T> struct Point {
    T x, y;
    Point(T x = 0, T y = 0) : x(x), y(y) {}
    Point operator+(const Point& p) const { return {x
+ p.x, y + p.y}; }
    Point operator-(const Point& p) const { return {x
- p.x, y - p.y}; }
    Point operator*(T c) const { return {x * c, y * c
}; }
    Point operator/(T c) const { return {x / c, y / c
}; }
    bool operator==(const Point& p) const { return x
== p.x && y == p.y; }
    bool operator<(const Point& p) const { return x <
p.x || (x == p.x && y < p.y); } //
    lexicografico
};
/**
 * Uso: dot(a, b), cross(a, b), cross(p, a, b),
    norm2(p);

```

```

 * Operaciones que devuelven el mismo tipo T del
    punto: producto punto, producto cruz (escalar)
    y norma al cuadrado.
 * Complejidad:  $O(1)$ .
 */
template<typename T> T dot(Point<T> a, Point<T> b)
{ return a.x * b.x + a.y * b.y; }
template<typename T> T cross(Point<T> a, Point<T> b
) { return a.x * b.y - a.y * b.x; }
template<typename T> T cross(Point<T> p, Point<T> a
, Point<T> b) { return cross(a - p, b - p); }
template<typename T> T norm2(Point<T> p) { return
dot(p, p); }

/**
 * Uso: norm(p);
 * Norma (longitud) de un punto o vector; devuelve
    long double.
 * Complejidad:  $O(1)$ .
 */
template<typename T> 1d norm(Point<T> p) { return
sqrt1(norm2(p)); }

/**
 * Uso: sgn(v);
 * Signo de un valor: devuelve 1 si  $v > 0$ , -1 si
     $v < 0$  y 0 si  $v = 0$ .
 * Complejidad:  $O(1)$ .
 */
template<typename T> int sgn(T v) { return (T(0) <
v) - (v < T(0)); }

/**
 * Uso: orient(a, b, c);
 * Orientacion del giro  $a \rightarrow b \rightarrow c$ : devuelve +1 si es
    antihorario (izquierda), -1 si es horario (
    derecha) y 0 si son colineales.
 * Complejidad:  $O(1)$ .
 */
template<typename T> int orient(Point<T> a, Point<T>
b, Point<T> c) { return sgn(cross(a, b, c)); }

/**
 * Uso: on_segment(p, a, b);
 * Devuelve true si el punto p pertenece al
    segmento [a,b] (incluyendo los extremos).
 * Complejidad:  $O(1)$ .
 */
template<typename T> bool on_segment(Point<T> p,
Point<T> a, Point<T> b) {
    return orient(a, b, p) == 0 && min(a.x, b.x) <= p
.x && p.x <= max(a.x, b.x) &&
    min(a.y, b.y) <= p.y && p.y <= max(a.y, b
.y);
}

/**
 * Uso: segment_intersect(a, b, c, d);
 * Devuelve true si los segmentos [a,b] y [c,d] se
    intersectan (incluye tocarse en un punto o
    superponerse).
 * Complejidad:  $O(1)$ .
 */
template<typename T> bool segment_intersect(Point<T>
a, Point<T> b, Point<T> c, Point<T> d) {
    int o1 = orient(a, b, c), o2 = orient(a, b, d),
    o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 != o2 && o3 != o4) return true;
    if (o1 == 0 && on_segment(c, a, b)) return true;

```

```

    if (o2 == 0 && on_segment(d, a, b)) return true;
    if (o3 == 0 && on_segment(a, c, d)) return true;
    if (o4 == 0 && on_segment(b, c, d)) return true;
    return false;
}

/**
 * Uso: polygon_area_2(poly);
 * Devuelve el doble del area del poligono (para
   evitar floats); el area real es area/2. Vertices
   en orden horario o antihorario.
 * Complejidad:  $O(N)$ .
 */
template<typename T> T polygon_area_2(const vector<
    Point<T>>& p) {
    T area = 0;
    for (int i = 0; i < p.size(); i++) area += cross(
        p[i], p[(i + 1) % p.size()]);
    return abs(area);
}

/**
 * Uso: boundary_points(poly);
 * Devuelve (long long) la cantidad de puntos del
   reticulo sobre el borde de un poligono (teorema
   de Pick:  $2A = 2I + B - 2$ ).
 * Complejidad:  $O(N)$ .
 */
long long boundary_points(const vector<Point<ll>>&
    poly) {
    ll b = 0;
    for (int i = 0; i < poly.size(); i++) {
        auto p1 = poly[i], p2 = poly[(i + 1) % poly.
            size()];
        b += gcd(abs(p1.x - p2.x), abs(p1.y - p2.y));
    }
    return b;
}

/**
 * Uso: point_in_polygon(poly, p);
 * Clasifica un punto respecto a un poligono:
   devuelve 0 fuera, 1 en el borde, 2 dentro.
 * Complejidad:  $O(N)$ .
 */
template<typename T> int point_in_polygon(const
    vector<Point<T>>& poly, Point<T> p) {
    int hits = 0;
    for (int i = 0; i < poly.size(); i++) {
        auto a = poly[i], b = poly[(i + 1) % poly.size()];
        if (on_segment(p, a, b)) return 1;
        if (a.y > b.y) swap(a, b);
        if (p.y >= a.y && p.y < b.y && cross(p, a, b) >
            0) hits++;
    }
    return hits & 1 ? 2 : 0;
}

/**
 * Uso: auto hull = convex_hull(puntos);
 * Cascara convexa (monotonic chain) en sentido
   antihorario, preservando puntos colineales en
   el borde; devuelve vector<Point<T>>.
 * Complejidad:  $O(N \log N)$ .
 */
template<typename T> vector<Point<T>> convex_hull(
    vector<Point<T>> pts) {
    int n = pts.size(), k = 0;

```

```

    if (n <= 2) return pts;
    vector<Point<T>> h(2 * n);
    sort(all(pts));
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(h[k - 2], h[k - 1], pts[
            i]) < 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(h[k - 2], h[k - 1], pts[
            i]) < 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}

/**
 * Uso: closest_pair(puntos);
 * Devuelve (long long) la distancia al cuadrado
   entre el par de puntos mas cercanos.
 * Complejidad:  $O(N \log N)$ .
 */
long long closest_pair(vector<Point<ll>> pts) {
    int n = pts.size();
    if (n < 2) return 8e18;
    sort(all(pts));
    set<pair<ll, ll>> active; // {y, x}
    ll best = 8e18;
    int left = 0;
    for (int i = 0; i < n; i++) {
        ll d = ceil(sqrt(1(best)));
        while (left < i && pts[i].x - pts[left].x >= d)
            active.erase({pts[left].y, pts[left].x}), left
                ++;
        auto lo = active.lower_bound({pts[i].y - d, -8
            e18});
        auto hi = active.upper_bound({pts[i].y + d, 8
            e18});
        for (auto it = lo; it != hi; ++it) {
            ll dy = pts[i].y - it->fi, dx = pts[i].x - it
                ->se;
            best = min(best, dx * dx + dy * dy);
        }
        active.insert({pts[i].y, pts[i].x});
    }
    return best;
}

/**
 * Uso: Event1D ev = {x, type, id};
 * Evento de sweep line 1D: type +1 marca el inicio
   (entrada) de un rango en x y type -1 su fin (
   salida); id identifica al intervalo.
 * Al ordenar por x se da prioridad a las entradas:
   en empate de x, primero el de mayor type (+1
   antes que -1).
 * Complejidad:  $O(1)$  por evento,  $O(K \log K)$  al
   ordenarlos con K eventos.
 */
struct Event1D {
    ll x;
    int type, id;
    bool operator<(const Event1D& o) const { return x
        != o.x ? x < o.x : type > o.type; }
};

```

```

/**
 * Uso: Event2D ev = {x, y1, y2, type};
 * Evento de sweep line 2D para rectangulos: cubre
   el rango vertical [y1,y2] (ambos extremos
   inclusivos); type +1 borde izquierdo (entrada)
   y -1 borde derecho (salida).
 * Al ordenar por x se da prioridad a las entradas:
   en empate de x, primero el de mayor type (+1
   antes que -1).
 * Complejidad: O(1) por evento, O(K log K) al
   ordenarlos con K eventos.
 */
struct Event2D {
    ll x, y1, y2;
    int type; // +1 borde izq, -1 borde der
    bool operator<(const Event2D& o) const { return x
        != o.x ? x < o.x : type > o.type; }
};

/**
 * Uso: polar_sort(puntos, centro);
 * Ordena los puntos en su lugar por angulo polar
   alrededor de un centro, en sentido antihorario
   desde el eje +x.
 * Complejidad: O(N log N).
 */
template<typename T> void polar_sort(vector<Point<T>
    >& pts, Point<T> center = {}) {
    auto half = [] (Point<T> p) { return p.y > 0 || (p
        .y == 0 && p.x > 0) ? 1 : -1; };
    sort(all(pts), [&] (Point<T> a, Point<T> b) {
        auto va = a - center, vb = b - center;
        int ha = half(va), hb = half(vb);
        if (ha != hb) return ha > hb;
        T cr = cross(va, vb);
        if (cr != 0) return cr > 0;
        return norm2(va) < norm2(vb);
    });
}

/**
 * Uso: dist_to_line(p, a, b);
 * Devuelve la distancia (long double) del punto p
   a la linea infinita que pasa por a y b.
 * Complejidad: O(1).
 */
template<typename T> ld dist_to_line(Point<T> p,
    Point<T> a, Point<T> b) {
    return abs((ld)cross(p, a, b)) / norm(a - b);
}

/**
 * Uso: dist_to_segment(p, a, b);
 * Devuelve la distancia (long double) del punto p
   al segmento [a,b].
 * Complejidad: O(1).
 */
template<typename T> ld dist_to_segment(Point<T> p,
    Point<T> a, Point<T> b) {
    if (a == b) return norm(p - a);
    if (dot(p - a, b - a) <= 0) return norm(p - a);
    if (dot(p - b, a - b) <= 0) return norm(p - b);
    return dist_to_line(p, a, b);
}

/**
 * Uso: line_intersect(a, b, c, d);
 * Devuelve el punto de interseccion (Point<long
   double>) de las lineas infinitas AB y CD; no es
   valido si son paralelas.
 * Complejidad: O(1).
 */
Point<ld> line_intersect(Point<ld> a, Point<ld> b,
    Point<ld> c, Point<ld> d) {
    ld cr1 = cross(b - a, c - a), cr2 = cross(b - a,
        d - a);
    return c + (d - c) * (cr1 / (cr1 - cr2));
}

/**
 * Uso: rotating_calipers(hull);
 * Devuelve el diametro al cuadrado de un poligono
   convexo mediante calipers rotativos (mismo tipo
   T del punto).
 * Complejidad: O(N).
 */
template<typename T> T rotating_calipers(const
    vector<Point<T>>& poly) {
    int n = poly.size();
    if (n < 2) return 0;
    if (n == 2) return norm2(poly[0] - poly[1]);
    T best = 0;
    int j = 1;
    for (int i = 0; i < n; i++) {
        int ni = (i + 1) % n;
        while (abs(cross(poly[j], poly[i], poly[ni])) <
            abs(cross(poly[(j + 1) % n], poly[i], poly[ni]
                )))) j = (j + 1) % n;
        best = max({best, norm2(poly[i] - poly[j]),
            norm2(poly[ni] - poly[j])});
    }
    return best;
}

/**
 * Uso: Circle c(centro, radio);
 * Circulo con centro Point<long double> y radio
   long double.
 * Complejidad: O(1).
 */
struct Circle {
    Point<ld> c;
    ld r;
    Circle(Point<ld> c = {}, ld r = 0) : c(c), r(r)
    {}
};

/**
 * Uso: circle_line_intersect(a, b, circ);
 * Devuelve vector<Point<long double>> con los 0, 1
   o 2 puntos donde la linea AB corta al circulo
   .
 * Complejidad: O(1).
 */
vector<Point<ld>> circle_line_intersect(Point<ld> a
    , Point<ld> b, Circle circ) {
    auto dir = b - a;
    ld len2 = norm2(dir);
    if (cmp(len2) == 0) return {};
    ld t = dot(circ.c - a, dir) / len2;
    auto proj = a + dir * t;
    ld d2 = norm2(circ.c - proj);
    if (cmp(d2, circ.r * circ.r) > 0) return {};
    if (cmp(d2, circ.r * circ.r) == 0) return {proj};
    ld off = sqrtl(max((ld)0, circ.r * circ.r - d2) /
        len2);
}

```

```

    return {proj - dir * off, proj + dir * off};
}
/**
 * Uso: circle_circle_intersect(c1, c2);
 * Devuelve vector<Point<long double>> con los 0, 1
 * o 2 puntos donde se cortan dos circulos (vacío
 * si son concéntricos o no se tocan).
 * Complejidad: O(1).
 */
vector<Point<ld>> circle_circle_intersect(Circle c1
, Circle c2) {
    auto dir = c2.c - c1.c;
    ld d = norm(dir);
    if (cmp(d, c1.r + c2.r) > 0 || cmp(d, abs(c1.r -
        c2.r)) < 0 || cmp(d) == 0) return {};
    ld a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 *
        d);
    ld h = sqrt1(max((ld)0, c1.r * c1.r - a * a));
    auto p2 = c1.c + dir * (a / d);
    auto perp = Point<ld>(-dir.y, dir.x) * (h / d);
    if (cmp(h) == 0) return {p2};
    return {p2 + perp, p2 - perp};
}
/**
 * Uso: Halfplane hp(a, b);
 * Un semiplano se crea con dos puntos: el plano
 * válido es el que queda a la izquierda de la
 * recta orientada a → b (producto cruz positivo).
 * Complejidad: O(1) por semiplano.
 */
struct Halfplane {
    Point<ld> p, pq; // p = punto, pq = vector
    ld angle;
    Halfplane() {}
    Halfplane(Point<ld> a, Point<ld> b) : p(a), pq(b
        - a), angle(atan2(pq.y, pq.x)) {}
    bool out(Point<ld> r) const { return cross(pq, r
        - p) < -EPS; } // r afuera (lado derecho
        estricto)
    bool operator<(const Halfplane& e) const {
        if (abs(angle - e.angle) > EPS) return angle <
            e.angle;
        return cross(pq, e.p - p) > EPS;
    }
};
/**
 * Uso: auto p = intersect(hp1, hp2);
 * Devuelve el punto de corte (Point<long double>)
 * entre las rectas de dos semiplanos.
 * Complejidad: O(1).
 */
Point<ld> intersect(Halfplane s, Halfplane t) {
    ld alpha = cross(t.p - s.p, t.pq) / cross(s.pq, t
        .pq);
    return s.p + s.pq * alpha;
}
/**
 * Uso: auto poly = halfplane_intersection(H);
 * Intersección de un conjunto de semiplanos (cada
 * uno válido a la izquierda de su recta);
 * devuelve el polígono convexo resultante (vacío
 * si la región es ilimitada o no existe).
 * Complejidad: O(N log N).
 */

```

```

vector<Point<ld>> halfplane_intersection(vector<
    Halfplane>& H) {
    sort(all(H));
    int n = H.size(), k = 0;
    vector<Halfplane> U(n);
    for (int i = 0; i < n; i++) {
        if (i > 0 && abs(H[i].angle - H[i - 1].angle) <
            EPS) continue;
        U[k++] = H[i];
    }
    H = U; H.resize(k); n = k;
    deque<Halfplane> dq;
    dq.push_back(H[0]);
    if (n > 1) dq.push_back(H[1]);
    for (int i = 2; i < n; i++) {
        while (dq.size() >= 2 && H[i].out(intersect(dq.
            back(), dq[dq.size() - 2]))) dq.pop_back();
        while (dq.size() >= 2 && H[i].out(intersect(dq.
            front(), dq[1]))) dq.pop_front();
        dq.push_back(H[i]);
    }
    while (dq.size() >= 3 && dq.front().out(intersect
        (dq.back(), dq[dq.size() - 2]))) dq.pop_back();
    while (dq.size() >= 3 && dq.back().out(intersect(
        dq.front(), dq[1]))) dq.pop_front();
    vector<Point<ld>> poly;
    if (dq.size() < 3) return poly;
    for (int i = 0; i < dq.size(); i++) poly.pb(
        intersect(dq[i], dq[(i + 1) % dq.size()]));
    vector<Point<ld>> clean; // quita vertices
        duplicados y colineales
    for (int i = 0; i < poly.size(); i++) {
        auto cur = poly[i], nxt = poly[(i + 1) % poly.
            size()];
        if (hypot(cur.x - nxt.x, cur.y - nxt.y) < EPS)
            continue;
        if (!clean.empty()) {
            auto prv = clean.back();
            if (abs(cross(cur - prv, nxt - cur)) < EPS)
                continue;
        }
        clean.pb(cur);
    }
    return clean;
}
/**
 * Uso: Point3D p(x, y, z); p.dot(q); p.cross(q); p
 * .length();
 * Punto 3D con operadores aritmeticos, producto
 * punto, producto cruz y norma.
 * Complejidad: O(1).
 */
struct Point3D {
    double x, y, z;
    Point3D(double x = 0, double y = 0, double z = 0)
        : x(x), y(y), z(z) {}
    Point3D operator+(const Point3D& p) const {
        return {x + p.x, y + p.y, z + p.z};
    }
    Point3D operator-(const Point3D& p) const {
        return {x - p.x, y - p.y, z - p.z};
    }
    Point3D operator*(double c) const { return {x * c
        , y * c, z * c}; }
    double dot(const Point3D& p) const { return x * p
        .x + y * p.y + z * p.z; }
    Point3D cross(const Point3D& p) const { return {y

```

```

    * p.z - z * p.y, z * p.x - x * p.z, x * p.y -
    y * p.x}; }
    double length() const { return sqrt(dot(*this));
    }
};
/**
 * Uso: dist_point_plane(p, a, n), dist_point_line(
 * p, a, b);
 * Distancia (double) de un punto a un plano (dado
 * por el punto a y la normal n) y a una línea (
 * por los puntos a y b).
 * Complejidad: O(1).
 */
double dist_point_plane(Point3D p, Point3D a,
    Point3D n) { return abs((p - a).dot(n)) / n.
    length(); }
double dist_point_line(Point3D p, Point3D a,
    Point3D b) { return (p - a).cross(b - a).length
    () / (b - a).length(); }
/**
 * Uso: line_plane_intersect(L1, L2, P, N);
 * Punto de interseccion (Point3D) de la línea
 * L1 → L2 con el plano definido por el punto P y
 * la normal N; devuelve (109, 109, 109) si son
 * paralelos.
 * Complejidad: O(1).
 */
Point3D line_plane_intersect(Point3D L1, Point3D L2
    , Point3D P, Point3D N) {
    auto u = L2 - L1;
    double d = N.dot(u);
    if (abs(d) > EPS) return L1 + u * ((P - L1).dot(N)
        ) / d);
    return {1e9, 1e9, 1e9}; // paralelos
}

```

Bit Operations

```

/**
 * Uso: check_bit(m, i), set_bit(m, i), clear_bit(m
 * , i), toggle_bit(m, i), lsb(m). i 0-based.
 * Manipulacion basica de bits de una mascara:
 * consultar, activar, limpiar, alternar y aislar
 * el bit menos significativo.
 * Complejidad: O(1) por operacion.
 */
inline bool check_bit(ll mask, int i) { return (
    mask >> i) & 1; }
inline ll set_bit(ll mask, int i) { return mask |
    (1LL << i); }
inline ll clear_bit(ll mask, int i) { return mask &
    ~(1LL << i); }
inline ll toggle_bit(ll mask, int i) { return mask
    ^ (1LL << i); }
inline ll lsb(ll mask) { return mask & -mask; }
/**
 * Uso: count_bits(x), count_bits_ll(x),
 * highest_bit(x) = bit mas alto (-1 si 0), ctz(x)
 * = ceros a la derecha.
 * Conteo de bits activos, posicion del bit mas
 * alto y numero de ceros finales con instrucciones
 * nativas.
 * Complejidad: O(1) por operacion.
 */
inline int count_bits(int mask) { return
    __builtin_popcount(mask); }

```

```

inline int count_bits_ll(ll mask) { return
    __builtin_popcountll(mask); }
inline int highest_bit(ll mask) { return mask == 0
    ? -1 : 63 - __builtin_clzll(mask); }
inline int ctz(ll mask) { return mask == 0 ? 64 :
    __builtin_ctzll(mask); }
/**
 * Uso: iterate_submasks(m);
 * Itera todas las sub-mascaras de una mascara dada
 * en orden decreciente (sin incluir la vacia).
 * Complejidad: O(3N) en total para una mascara de
 * N bits.
 */
inline void iterate_submasks(ll mask) {
    for (ll sub = mask; sub; sub = (sub - 1) & mask)
        { /* procesar sub */ }
}
/**
 * Uso: auto masks = gopers_hack(k, n);
 * Genera todas las mascarar de tamaño n con
 * exactamente k bits activos (gopers hack).
 * Complejidad: O( $\binom{n}{k}$ ) mascarar generadas, O(1)
 * amortizado por mascara.
 */
inline vector<ll> gopers_hack(int k, int n) {
    vector<ll> res;
    if (!k) return {0};
    ll mask = (1LL << k) - 1, limit = 1LL << n;
    while (mask < limit) {
        res.pb(mask);
        ll c = mask & -mask, r = mask + c;
        mask = (((r ^ mask) >> 2) / c) | r;
    }
    return res;
}
/**
 * Uso: XorBasis b; b.insert(x); b.can_form(k); b.
 * get_max(init); b.get_min();
 * Base lineal de xor sobre enteros de 64 bits:
 * maximo xor de un subconjunto, minimo y test de
 * representabilidad.
 * Complejidad: insercion y consultas O(60).
 */
struct XorBasis {
    static const int BITS = 60;
    ll basis[BITS] = {};
    int sz = 0;
    // Inserta; true si fue linealmente independiente
    bool insert(ll x) {
        for (int i = BITS - 1; i >= 0; i--) if (
            check_bit(x, i)) {
            if (!basis[i]) return basis[i] = x, sz++,
                true;
            x ^= basis[i];
        }
        return false;
    }
    bool can_form(ll k) {
        for (int i = BITS - 1; i >= 0; i--) if (
            check_bit(k, i)) {
            if (!basis[i]) return false;
            k ^= basis[i];
        }
        return true;
    }
}

```

```

    }
    // Maximo de (initial ^ subset)
    ll get_max(ll initial = 0) {
        for (int i = BITS - 1; i >= 0; i--) if ((
            initial ^ basis[i]) > initial) initial ^= basis
            [i];
        return initial;
    }
    ll get_min() {
        for (int i = 0; i < BITS; i++) if (basis[i])
            return basis[i];
        return 0;
    }
};

/**
 * Uso: BitTrie t; t.insert(x); t.max_xor(x); t.
 * insert(x, -1) borra.
 * Trie binario para consultar el maximo xor de un
 * valor contra el conjunto insertado, con soporte
 * de borrado.
 * Complejidad: insercion y consulta  $O(B)$  con  $B = 30$ 
 * bits (60 si los valores llegan a  $10^{18}$ ).
 */
struct BitTrie {
    static const int BITS = 30; // 60 si los numeros
    llegan a  $1e18$ 
    struct Node { int next[2] = {-1, -1}; int cnt =
        0; };
    vector<Node> t;
    BitTrie() { t.emplace_back(); }
    void insert(ll x, int val = 1) { // val=-1 para
        borrar
        int u = 0;
        for (int i = BITS - 1; i >= 0; i--) {
            int b = check_bit(x, i);
            if (t[u].next[b] == -1) t[u].next[b] = t.size
            (), t.emplace_back();
            u = t[u].next[b];
            t[u].cnt += val;
        }
    }
    ll max_xor(ll x) {
        int u = 0; ll ans = 0;
        for (int i = BITS - 1; i >= 0; i--) {
            int b = check_bit(x, i), opp = b ^ 1;
            int v = t[u].next[opp];
            if (v != -1 && t[v].cnt > 0) ans |= 1LL << i,
                u = v;
            else u = t[u].next[b];
        }
        return ans;
    }
};

```

Extra

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

/**
 * Uso[0]: ordered_set<T> s; s.insert(x); s.
 * order_of_key(x); *s.find_by_order(k);
 * Conjunto ordenado de pbds con consultas por
 * rango (k-esimo menor y posicion).
 * Complejidad:  $O(\log N)$  por operacion.
 */

```

```

template <typename T>
using ordered_set = __gnu_pbds::tree<T, __gnu_pbds
::null_type, less<T>, __gnu_pbds::rb_tree_tag,
__gnu_pbds::tree_order_statistics_node_update>;

/**
 * Uso[0]: ordered_multiset<T> s; s.insert({x, id})
 * ;
 * Conjunto ordenado de pbds que admite duplicados
 * emparejando cada valor con un id unico.
 * Complejidad:  $O(\log N)$  por operacion.
 */
template <typename T>
using ordered_multiset = __gnu_pbds::tree<pair<T,
int>, __gnu_pbds::null_type, less<pair<T, int
>>, __gnu_pbds::rb_tree_tag, __gnu_pbds::
tree_order_statistics_node_update>;

/**
 * Uso: unordered_map<ll, int, custom_hash> m;
 * Functor de hash anti-colision (splitmix64 con
 * semilla aleatoria) para tablas hash de claves
 * numericas.
 * Complejidad:  $O(1)$  por operacion de hash.
 */
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::
        steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

/**
 * Uso: MeetInTheMiddle::subset_sum(a, target);
 * Resuelve subset sum exacto con la tecnica meet
 * in the middle para arreglos de hasta 40
 * elementos.
 * Complejidad:  $O(N \cdot 2^{N/2})$  tiempo,  $O(2^{N/2})$  espacio.
 */
struct MeetInTheMiddle {
    static bool subset_sum(const vll& a, ll target) {
        int n = a.size(), mid = n / 2;
        vll left, right;
        for (int i = 0; i < (1 << mid); i++) {
            ll s = 0;
            for (int j = 0; j < mid; j++) if (i >> j & 1)
                s += a[j];
            left.pb(s);
        }
        int n2 = n - mid;
        for (int i = 0; i < (1 << n2); i++) {
            ll s = 0;
            for (int j = 0; j < n2; j++) if (i >> j & 1)
                s += a[mid + j];
            right.pb(s);
        }
        sort(all(right));
        for (ll s : left) if (binary_search(all(right),
            target - s)) return true;
        return false;
    }
};

```

```

/**
 * Uso[0]: ZobristHash zh; auto pref = zh.
 * build_pref(a); zh.query_range(pref, l, r);
 * Hash de prefijos xor para comparar multiconjuntos
 * de rangos en  $O(1)$ .
 * Complejidad: construccion  $O(N)$ , consulta  $O(1)$ .
 */
struct ZobristHash {
    using u64 = uint64_t;
    mt19937_64 rng;
    unordered_map<int, u64> val_hash;
    ZobristHash(u64 seed = 1337) : rng(seed) {
        val_hash[0] = 0; }
    u64 get_hash(int x) {
        if (!val_hash.count(x)) val_hash[x] = rng();
        return val_hash[x];
    }
    vector<u64> build_pref(const vi& a) { // prefijos
        XOR
        vector<u64> pref(a.size() + 1, 0);
        for (int i = 0; i < a.size(); i++) pref[i + 1]
            = pref[i] ^ get_hash(a[i]);
        return pref;
    }
    u64 query_range(const vector<u64>& pref, int l,
        int r) { return pref[r + 1] ^ pref[l]; }
};

/**
 * Uso[1]: BitsetReachability br(n); br.add_edge(u,
 * v); br.build(); br.can_reach(u, v);
 * Cierre transitivo (alcanzabilidad todos-pares)
 * de un grafo dirigido usando bitsets de palabras
 * de 64 bits; el tamaño se fija en el constructor
 * .
 * Indexación: nodos 1-indexados.
 * Complejidad: construccion  $O(N^3/64)$ , consulta  $O(1)$ 
 * .
 */
struct BitsetReachability {
    int n, W;

```

```

    vector<vector<unsigned long long>> reach;
    BitsetReachability(int n) : n(n), W((n + 63) /
        64), reach(n + 1, vector<unsigned long long>(W,
            0)) {}
    void add_edge(int u, int v) { reach[u][v >> 6] |=
        1ULL << (v & 63); }
    void build() {
        for (int i = 1; i <= n; i++) reach[i][i >> 6]
            |= 1ULL << (i & 63);
        for (int k = 1; k <= n; k++)
            for (int i = 1; i <= n; i++)
                if ((reach[i][k >> 6] >> (k & 63)) & 1ULL)
                    for (int w = 0; w < W; w++) reach[i][w]
                        |= reach[k][w];
    }
    bool can_reach(int u, int v) { return (reach[u][v
        >> 6] >> (v & 63)) & 1ULL; }
};

/**
 * Uso[0]: auto [vals, ids] = compress_coords(a);
 * Compresión de coordenadas: ordena y elimina
 * duplicados (sort + unique) y mapea cada valor a
 * su rango consecutivo  $0..K-1$ .
 * Indexación: ids 0-indexados en orden creciente
 * de valor.
 * Complejidad:  $O(N \log N)$  tiempo,  $O(N)$  espacio.
 */
template<typename T>
pair<vector<T>, vector<int>> compress_coords(const
    vector<T>& a) {
    vector<T> vals = a;
    sort(all(vals));
    vals.erase(unique(all(vals)), vals.end());
    vector<int> ids(a.size());
    for (int i = 0; i < (int)a.size(); i++)
        ids[i] = lower_bound(all(vals), a[i]) - vals.
            begin();
    return {vals, ids};
}

```